

普中 STM32F1-UCOSIII 开发攻略



开源共享

携手共进

深圳普中科技有限公司

官方网站: www.prechin.cn

技术论坛: www.prechin.net

目录

普中 STM32F1-UCOSIII 开发攻略.....	1
目录.....	2
第 1 章 UCOSIII 移植.....	1
1.1 UCOSIII 简介.....	2
1.1.1 UCOSIII 简介.....	2
1.1.2 UCOSIII 特点.....	2
1.1.3 UCOS、UCOSII、UCOSIII 的性能对比.....	7
1.2 UCOSIII 移植过程.....	8
1.2.1 UCOSIII 源码下载.....	8
1.2.2 UCOSIII 移植.....	14
1.2.3 修改 bsp.c 和 bsp.h 文件.....	26
1.2.4 修改 os_cpu_a.asm 文件.....	29
1.2.5 修改 os_cfg_app.h 文件.....	31
1.2.6 修改 Public 文件夹内文件.....	32
1.3 实验现象.....	44
课后作业.....	44

第 1 章 UCOSIII 移植

本章将向大家介绍 UCOSIII 移植，通过本章的学习，让大家能够快速将 UCOSIII 移植到 STM32F1 上。本章分为如下几部分内容：

- 1.1 UCOSIII 简介
- 1.2 UCOSIII 移植过程
- 1.3 实验现象

普中科技STM32开发板

1.1 UCOSIII 简介

1.1.1 UCOSIII 简介

uC/OS-III (Micro C OS Three 微型的 C 语言编写的操作系统第 3 版) 是一个可升级的, 可固化的, 基于优先级的实时内核。它对任务的个数无限制。uC/OS-III 是一个第 3 代的系统内核, 支持现代的实时内核所期待的大部分功能。例如资源管理, 同步, 任务间的通信等等。然而, uC/OS-III 提供的特色功能在其它的实时内核中是找不到的, 比如说完备的运行时间测量性能, 直接地发送信号或者消息到任务, 任务可以同时等待多个内核对象等。

uC/OS-III 是一个可扩展的, 可固化的, 抢占式的实时内核, 它管理的任务个数不受限制。它是第三代内核, 提供了现代实时内核所期望的所有功能包括资源管理、同步、内部任务交流等。uC/OS-III 也提供了很多特性是在其他实时内核中所没有的。比如能在运行时测量运行性能, 直接得发送信号或消息给任务, 任务能同时等待多个信号量和消息队列。

uC/OS 系列, 第一代产生于 1992。经过了多年的使用和上千人的反馈, 已经产生了很多的进化版本。uC/OS-III 是这些反馈和经验的总结。在 uC/OS-II 中很少使用的功能已经被删除或者被更新, 添加了更高效的功能和服务。其中最有用的功能应该是时间片轮转法 (round robin), 这个是 uC/OS-II 中不支持的, 但是现在已经是 uC/OS-III 的一个功能了。uC/OS-III 会提供新的功能以更好地适应新出现的处理器。特别的, uC/OS-III 被设计用于 32 位处理器, 但是它也能在 16 位或 8 位处理器中很好地工作。

1.1.2 UCOSIII 特点

UCOSIII 特点很多, 主要如下:

(1) 源代码

uC/OS-III 完全根据 ANSI-C 标准写的。代码的规范是 Micrium 团队的一种文化。虽然很多商业内核供应商提供他们产品的源代码, 但是这些产品很有可能是笨重且难以利用的。除非代码严格地遵循标准并且产品有完整的带例子的说明

书以展示代码是怎样工作的。

(2) 应用程序接口 (API)

uC/OS-III 是很直观的。如果你熟悉类似的编码规范，你能轻松地知道函数名所对应的服务，以及需要怎样的参数。例如：指向对象的指针通常是第一个参数，指向错误代码的指针通常是最后一个参数。

(3) 抢占式多任务处理

uC/OS-III 是一个抢占式多任务处理内核，因此，uC/OS-III 正在运行的经常是最重要的就绪任务。

(4) 时间片轮转调度

uC/OS-III 允许多个任务拥有相同的优先级。当多个相同优先级的任务就绪时，并且这个优先级是目前最高的。uC/OS-III 会分配用户定义的时间片给每个任务去运行。每个任务可以定义不同的时间片。当任务用不完时间片时可以让出 CPU 给另一个任务。

(5) 快速响应中断

uC/OS-III 有一些内部的数据结构和变量。uC/OS-III 保护临界段可以通过锁定调度器代替关中断。因此关中断的时间会非常少。这样就使 uC/OS-III 可以响应一些非常快的中断源了。

(6) 确定性

uC/OS-III 的中断响应时间是可确定的，uC/OS-III 提供的大部分服务的执行时间也是可确定的。

(7) 可扩展

根据应用的需求，代码大小可以被调整。编译时通过调整 uC/OS-III 源代码中的大约 40 个 #define (见 OS_CFG.H) 可以在添加或移除一些功能。uC/OS-III 的服务还提供一些实时检查功能。特别的，uC/OS-III 能检查传递的参数是否为 NULL 指针，ISR 是否就绪了任务级服务。参数有允许范围，指定选项都是有用的。检测功能可以被关闭（在编译时）以提供更好的性能和缩减代码大小。实际上，可扩展的 uC/OS-III 支持更广泛的应用和项目。

(8) 易移植

uC/OS-III 可以被移植到大部分的 CPU 架构中。大部分的支持 uC/OS-II 的

器件通过改动就能支持 uC/OS-III。而 uC/OS-II 已经移植到 45 种 CPU 架构中了。

(9) 可固化

uC/OS-III 专为嵌入式系统设计，它可以跟应用程序代码一起被固化。

(10) 可实时配置

uC/OS-III 允许用户在运行时配置内核。特别的，所有的内核对象如任务、堆栈、信号量、事件标志组、消息队列、消息、互斥信号量、内存分区、软件定时器等都是在运行时分配的，以免在编译时的过度分配。

(11) 任务数无限制

uC/OS-III 对任务数量无限制。实际上，任务的数量限制于处理器能提供的内存大小。每一个任务需要有自己的堆栈空间，uC/OS-III 在运行时监控任务堆栈的生长。uC/OS-III 对任务的大小无限制。

(12) 优先级数无限制

uC/OS-III 对优先级的数量无限制。然而，配置 uC/OS-III 的优先级在 32 到 256 之间已经满足大多数的应用了。

(13) 内核对象数无限制

uC/OS-III 支持任何数量的任务、信号量、互斥信号量、事件标志组、消息队列、软件定时器、内存分区。用户在运行时分配所有的内核对象。

(14) 服务

uC/OS-III 提供了高档实时内核所需要的所有功能，例如任务管理、时间管理、信号量、事件标志组、互斥信号量、消息队列、软件定时器、内存分区等。

(15) 互斥信号量

互斥信号量用于资源管理。它是一个内置优先级的特殊类型信号量，用于消除优先级反转。互斥信号量可以被嵌套，因此，任务可申请同一个互斥信号量多达 250 次。当然，互斥信号量的占有者需要释放同等次数。

(16) 嵌套的任务停止

uC/OS-III 允许任务停止自身或者停止另外的任务。停止一个任务意味着这个任务将不再执行直到被其他的任务恢复。停止可以被嵌套到 250 级。换句话说，一个任务可以停止另外的任务多达 250 次。当然，这个任务必须被恢复同

等次数才有资格再次获得 CPU。

(17) 软件定时器

可以定义任意数量的一次性的、周期性的、或者两者兼有的定时器。定时器是倒计时的，执行用户定义的行为一直到计数减为 0。每一个定时器可以有自己的行为，如果一个定时器是周期性的，计数减为 0 时会自动重装计数值并执行用户定义的行为。

(18) 挂起多个对象

uC/OS-III 允许任务等待多个事件的发生。特别的，任务可以同时等待多个信号量和消息队列被提交。等待中的任务在事件发生的时候被唤醒。

(19) 任务信号量

uC/OS-III 允许 ISR 或者任务直接地发送信号量给其它任务。这样就避免了必须产生一个中间级内核对象如一个信号量或者事件标志组只为了标记一个任务。提高了内核性能。

(20) 任务消息

uC/OS-III 允许 ISR 或者任务直接发送消息到另一个任务。这样就避免产生一个消息队列，提高了内核性能。

(21) 任务寄存器

每一个任务可以拥有用户可定义的任务寄存器，不同于 CPU 寄存器。

(22) 错误检测

uC/OS-III 能检测指针是否为 NULL、在 ISR 中调用的任务级服务是否允许、参数在允许范围内、配置选项的有效性、函数的执行结果等。每一个 uC/OS-III 的 API 函数返回一个对应于函数调用结果的错误代号。

(23) 内置的性能测量

uC/OS-III 有内置性能测量功能。能测量每一个任务的执行时间, 每个任务的堆栈使用情况, 任务的执行次数, CPU 的使用情况, ISR 到任务的切换时间, 任务到任务的切换时间, 列表中的峰值数, 关中断、锁调度器平均时间等。

(24) 可优化

uC/OS-III 被设计于能够根据 CPU 的架构被优化。uC/OS-III 所用的大部分数据类型能够被改变, 以更好地适应 CPU 固有的字大小。优先级调度法则可以

通过编写一些汇编语言而获益于一些特殊的指令如位设置、位清除、计数清零指令 (CLZ), find-first-one (FF1) 指令。

(25) 死锁预防

uC/OS-III 中所有的挂起服务都可以有时间限制, 预防死锁。

(26) 任务级的时基处理

uC/OS-III 有时基任务, 时基 ISR 触发时基任务。uC/OS-III 使用了哈希列表结构, 可以大大减少处理延时和任务超时所产生的开支。

(27) 用户可定义的钩子函数

uC/OS-III 允许程序员定义 hook 函数, hook 函数被 uC/OS-III 调用。hook 函数允许用户扩展 uC/OS-III 的功能。有的 hook 函数在任务切换的时候被调用, 有的在任务创建的时候被调用, 有的在任务删除的时候被调用。

(28) 时间戳

为了测量时间, uC/OS-III 需要一个 16 位或者 32 位的时时间戳计数器。这个计数器值可以在运行时被读取以测量时间。例如: 当 ISR 提交消息到任务时, 时间戳计数器自动读取并保存作为消息。当接收者接收到这条消息, 时间戳被提供在消息内。通过读取现在的时间戳, 消息的响应时间可以被确定。

(29) 嵌入的内核调试器

这个功能允许内核调试器查看 uC/OS-III 的变量和数据结构通过一个用户定义的通道。(但是只能在调试器遇到断点的时候查看)。uC/OS-III 内核也支持 uC/Probe (探针) 在运行时显示信息。

(30) 对象名称

每个 uC/OS-III 的内核对象有一个相关联的名字。这样就能很容易的识别出对象所指定的作用。分配一个 ASCII 码的名字给任务、信号量、互斥信号量、事件标志组、消息队列、内存块、软件定时器。对象的名字长度没有限制, 但是必须以空字符结束。

1.1.3 UCOS、UCOSII、UCOSIII 的性能对比

功能	uC/OS	uC/OS-II	uC/OS-III
诞生年份	1992	1998	2009
书	有	有	有
提供源代码	是	是	是
抢占式多任务	是	是	是
最大任务数	64	256	无限制
每个优先级的任务数	1	1	无限制
时间片轮转	否	否	是
信号量	是	是	是
互斥信号量	否	是	是（可嵌套的）
事件标志	否	是	是
消息邮箱	是	是	否（不需要了）
消息队列	是	是	是
固定大小的内存管理	否	是	是

不通过信号量标记一个任务	否	否	是
不通过消息队列发消息给任务	否	否	是
软件定时器	否	是	是
任务停止/恢复	否	是	是（可嵌套的）
死锁预防	是	是	是
可扩展的	是	是	是
代码段需求	3K 到 8K	6K 到 26K	6K 到 20K
数据段需求	1K+	1K+	1K+
可固化	是	是	是
在运行时配置	否	否	是
编译时配置	是	是	是
每个对象命名	否	是	是
挂起多个对象	否	是	是
任务寄存器	否	是	是

嵌入的测量功能	否	有限制	大量的
用户可定义的 hook 函数	否	是	是
时间戳	否	否	是
嵌入的内核调试	否	是	是
汇编可优化	否	否	是

任务级的时基定时器处理	否	否	是
提供的服务	~20	~90	~70
MISRA-C:1998	否	是（除了 10 个规则）	N/A
MISRA-C:2004	否	否	是（除了 7 个规则）
DO178B EUROCAE ED-12B	否	是	申请中
FDA 认证	否	是	申请中
SIL3/SIL4 IEC	否	是	申请中
IEC-61508	否	是	申请中

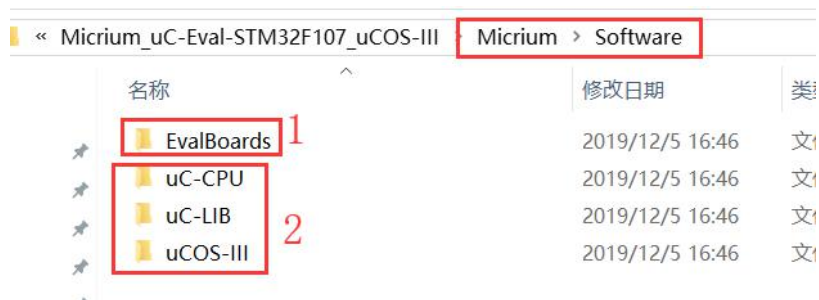
1.2 UCOSIII 移植过程

1.2.1 UCOSIII 源码下载

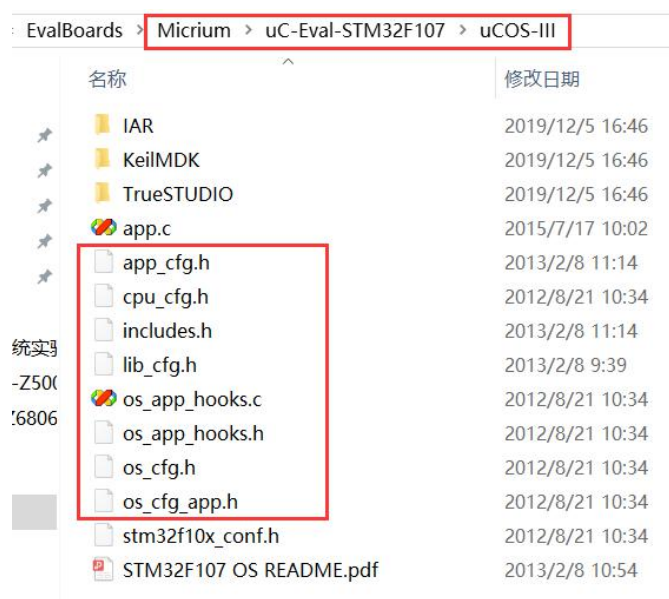
要将 UCOSIII 移植到我们 STM32F1 上，首先需到 Micrium 公司官方网站（<http://micrium.com/>）下载 UCOSIII 源码，这里我们已经将 Micrium 官方在 STM32F1xx 上移植好的工程下载好，放入光盘“\4--实验程序\3--RTOS 操作系统实验\UCOSIII 操作系统实验\官方 uCOS-III 源码”，省去了大家注册下载的时间。如图所示：

3--RTOS操作系统实验 > UCOSIII操作系统实验 > 官方 uCOS-III 源码			
名称	修改日期	类	
Micrium_uC-Eval-STM32F107_uCOS-III	2019/12/5 16:46	文	

打开该文件夹，如图所示（注意目录路径）：



(1) EvalBoards 文件夹：这个文件夹里面就是关于 STM32F1 的工程文件，其实我们直接可以使用，当然我们需要针对自己的开发板进行移植，打开这个文件夹，如图所示（注意目录路径）：

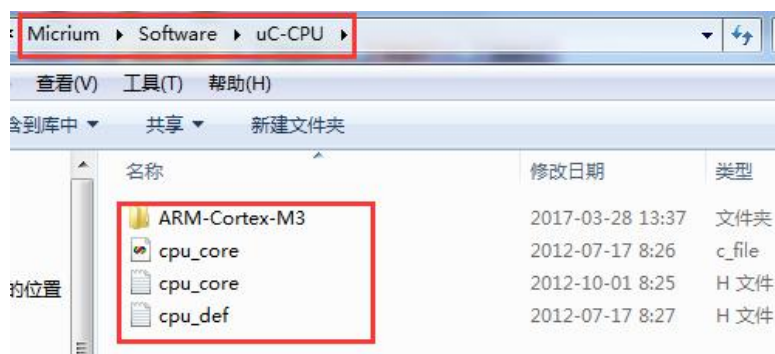


上图中红色框圈起来的是我们后面 UCOSIII 移植时需要添加到工程中的文件，这里一共 8 个文件。

(2) uC-CPU/uC-LIB/uCOS-III 文件夹，如下：



①uC-CPU 文件夹：与 CPU 相关的文件，打开此文件夹，如图所示（注意目录路径）：



1) cpu_core.c 文件

该文件包含了 CPU 初始化函数、CPU 名字、时间戳计算等 CPU 核心函数。

2) cpu_core.h 文件

CPU 核心配置，编译设置，cpu_core.c 相关函数声明、用来测量中断关闭时间变量的定义等。

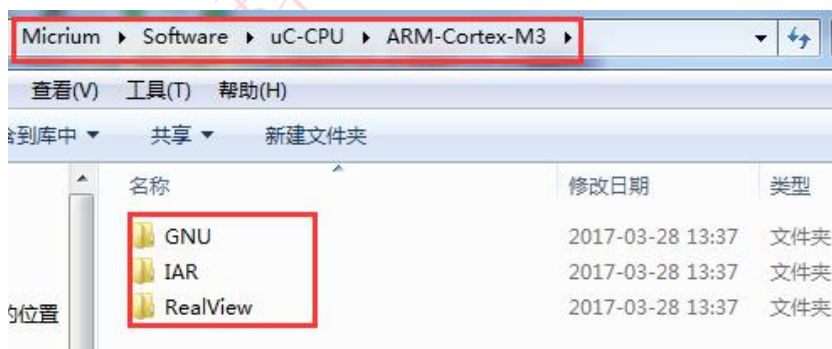
3) cpu_def.h 文件

CPU 相关配置，比如关中断的方式，堆栈增长方向，字长等。

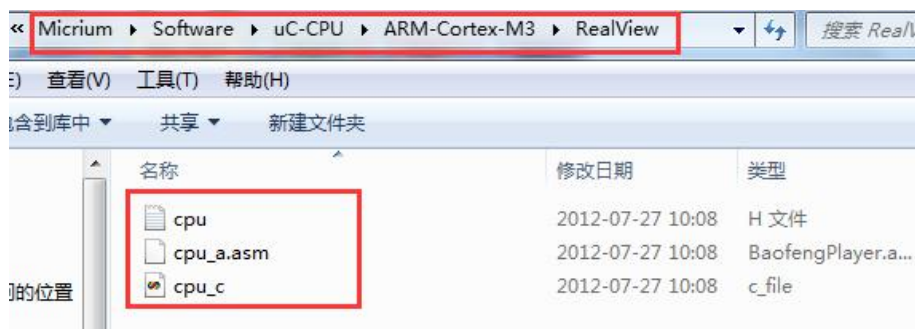
4) ARM-Cortex-M3 文件夹

CPU 移植文件，用户如果想要移植 uC/OS-III 到不同平台上，需要修改这部分代码。

此文件夹下有针对不同编译器的文件夹，如图所示（注意目录路径）：



我们使用的是 KEIL，所以移植的时候只需要添加“RealView”文件夹内的文件（后面的讲解中还会遇到同样的文件夹，需根据不同编译器选择文件），此文件夹内有 3 个文件，如图所示（注意目录路径）：



1) cpu.h 文件

包含了一些类型的定义,使 UCOSIII 和其他模块可与 CPU 架构和编译器字宽度无关。在该文件中用户能够找到 CPU_INT16U、CPU_INT32U、CPU_FP32 等数据类型的定义。该文件还指定了 CPU 使用的是大端模式还是小端模式,定义了 UCOSIII 使用的 CPU_STK 数据类型,定义了 CPU_CRITICAL_ENTER() 和 CPU_CRITICAL_EXTI(), 还包括一些与 CPU 架构相关的函数的声明。

2) cpu_a.asm 文件

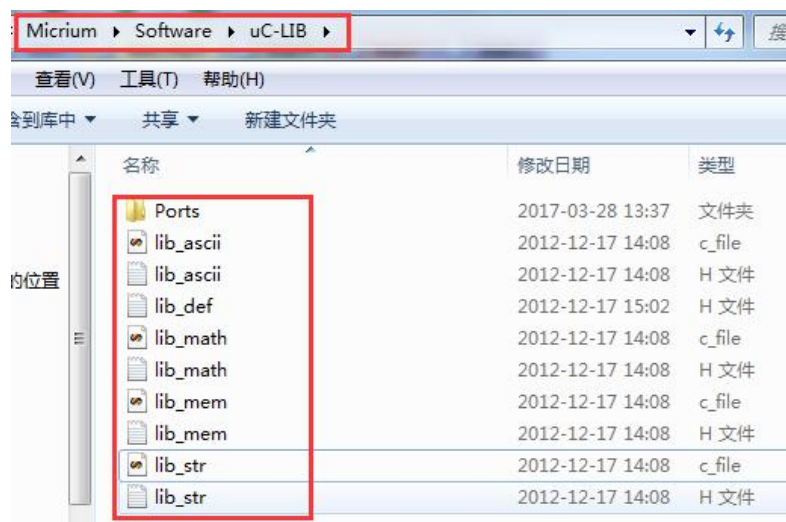
该文件包含了一些用汇编语言编写的函数,可用来开中断和关中断,计算前导零(如果 CPU 支持这条指令),以及其他一些只能用汇编语言编写的与 CPU 相关的函数,这个文件中的函数可以从 C 代码中调用。

3) cpu_c.c 文件

包含了一些基于特定 CPU 架构但为了可移植而用 C 语言编写的函数 C 代码,作为一个普通原则,除非汇编语言能显著提高性能,否则尽量用 C 语言编写函数。

②uC-LIB 文件夹

底层函数库,比如字符串的常规操作,常用的数学计算等等。该文件夹下有如下文件:



1) lib_ascii.h 和 lib_ascii.c 文件

提供 ASCII_ToLower()、ASCII_ToUpper()、ASCII_IsAlpha() 和 ASCII_IsDig() 等函数，它们可以分别替代标准库函数 tolower()、toupper()、isalpha() 和 isdigit() 等。

2) lib_def.h 文件

定义了许多常量，如 RTUE/FALSE、YES/NO、ENABLE/DISABLE，以及各种进制的常量。但是，该文件中所有#define 常量都以 DEF_打头，所以上述常量的名字实际上为 DEF_TRUE/DEF_FALSE、DEF_YES/DEF_NO、DEF_ENABLE/DEF_DISABLE 等。该文件还为常用数学计算定义了宏。

3) lib_math.h 和 lib_math.c 文件

包含了 Math_Rand()、Math_SetRand() 等函数的源代码，可用来替代标准库函数 rand()、srand()。

4) lib_mem.c 和 lib_mem.h 文件

包含了 Mem_Clr()、Mem_Set()、Mem_Copy() 和 Mem_Cmp() 等函数的源代码，可用来替代标准库函数 memclr()、memset()、memcpy() 和 memcmp() 等。

5) lib_str.c 和 lib_str.h 文件

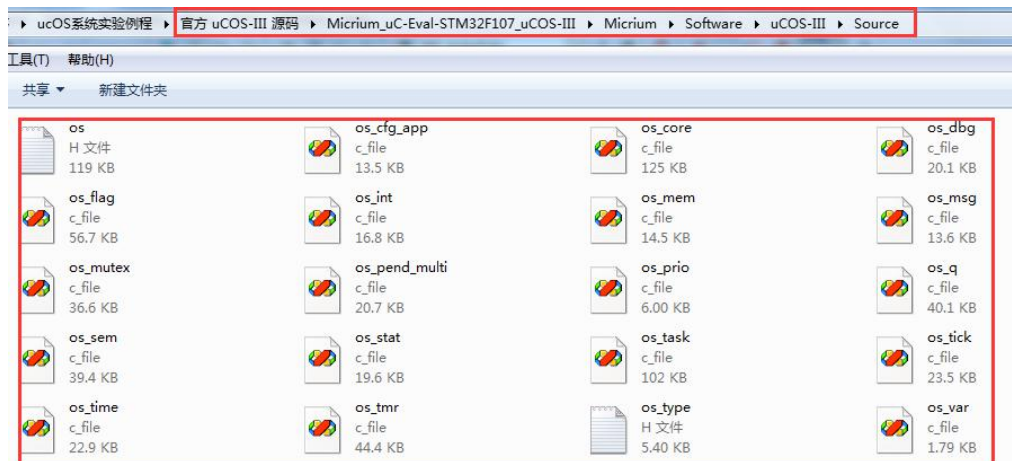
包含了 Str_Lenr()、Str_Copy() 和 Str_Cmp() 等函数的源代码，可用于替代标准库函数 strlen()、strcpy() 和 strcmp() 等。

6) lib_mem_a.asm 文件

lib_mem_a.asm 文件在 “Ports->ARM-Cortex-M3->RealView” 下，包含了 lib_mem.c 函数的汇编优化版。

③uCOS-III 文件夹

这个文件夹内有两个文件夹 Ports 和 Sourec, Ports 文件夹为与 CPU 平台有关的文件, Source 文件夹里面为 UCOSIII 3.03 的源码, 我们打开 Source 文件夹如图所示:



不同版本的源码基本是一样的, 不同的是各个文件里面有些函数作了修改, 各文件描述如图所示:

文件	描述
os.h	包含 UCOSIII 的主要头文件, 声明了常量、宏、全局变量、函数原型等
os_cfg_app.c	根据 os_cfg_app.h 中的宏定义声明变量和数组
os_core.c	UCOSIII 的内核功能模块
os_dbg.c	包含内核调试或 uC/Probe 使用的常量的声明
os_flag.c	包含事件标志的管理代码
os_int.c	包含中断处理任务的代码
os_mem.c	包含 UCOSIII 固定大小的存储分区的管理代码
os_msg.c	包含消息处理的代码
os_mutex.c	包含互斥型信号量的管理代码
os_pend_multi.c	包含允许任务同时等待多个信号量或多个消息队列的代码
os_prio.c	包含位映射表的管理代码, 用于追踪那些已经就绪的任务
os_q.c	包含消息队列的管理代码
os_sem.c	包含信号量的管理代码
os_stat.c	包含统计任务的代码
os_task.c	包含任务的管理代码
os_tick.c	包含可管理正在延时和超时等待的任务的代码
os_time.c	包含可是任务延时一段时间的代码
os_tmr.c	包含软件定时器的管理代码
os_type.h	包含 UCOSIII 数据类型的声明
os_var.c	包含 UCOSIII 的全局变量

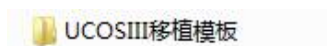
UCOSIII 源码就介绍到这里, 想要更加深入了解 UCOSIII 可以参考光盘“\8--STM32 相关资料\STM32 深入学习资料\UCOS 学习资料\UCOSIII 资料”中的《uCOS-III 中文翻译》文档, 该文档对 UCOSIII 讲解的非常详细, 接下来我们

开始将 UCOSIII 移植到我们的 STM32F1 上。

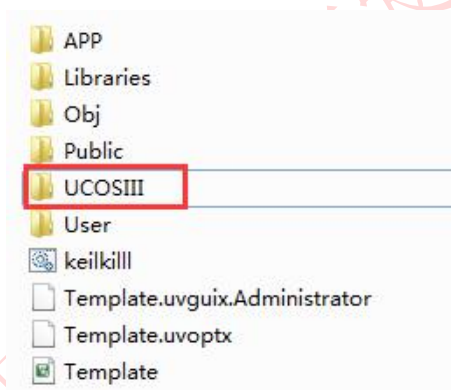
1.2.2 UCOSIII 移植

要将 UCOSIII 移植到 STM32F1 上，需要一个基础工程，在光盘“\4--实验程序\3--RTOS 操作系统实验\UCOSIII 操作系统实验\实验 1：移植 UCOSIII 所需的基础工程”中，其实这个就是我们基础实验的“14-printf 重定向实验”。移植具体步骤如下：

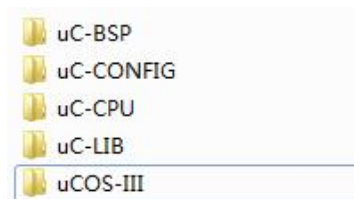
(1) 首先将基础工程程序复制一份，将文件夹命名为“UCOSIII 移植模板”，如图所示：



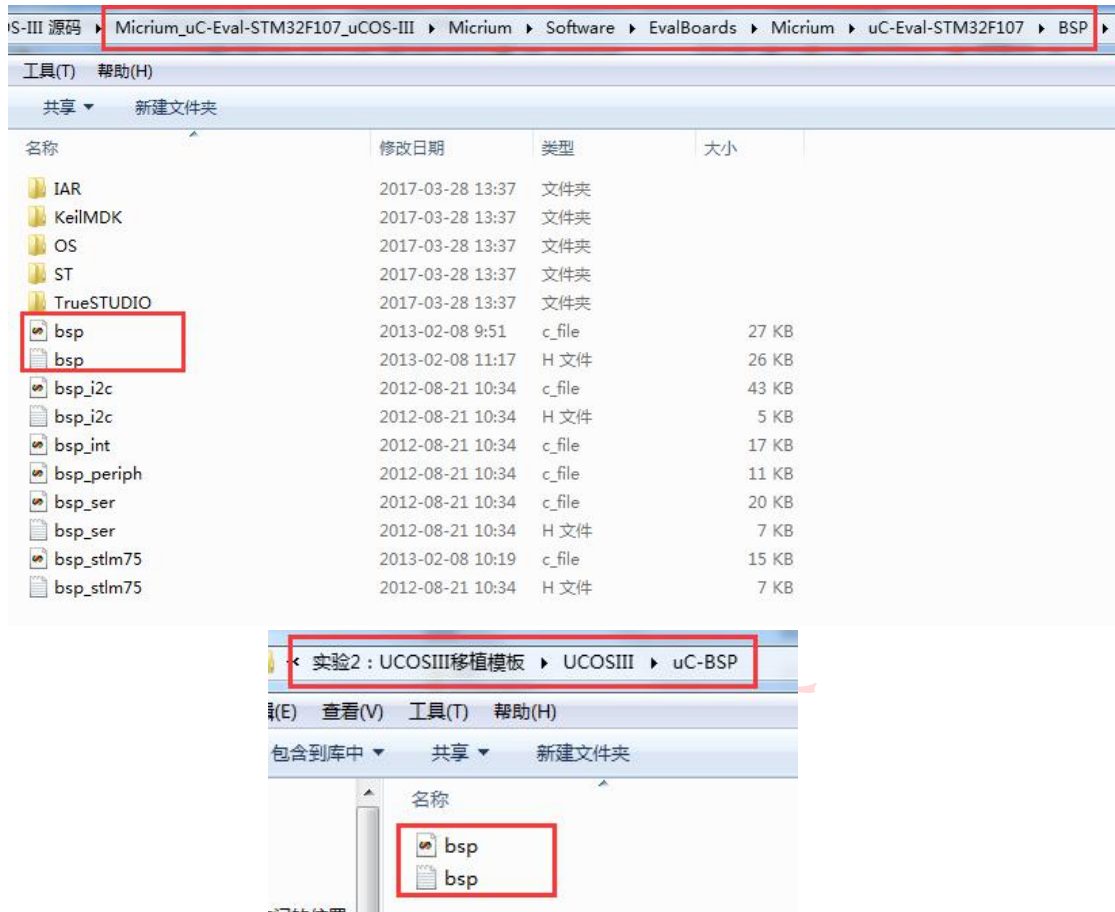
(2) 在该工程目录下新建一个文件夹，命名为“UCOSIII”，如图所示：



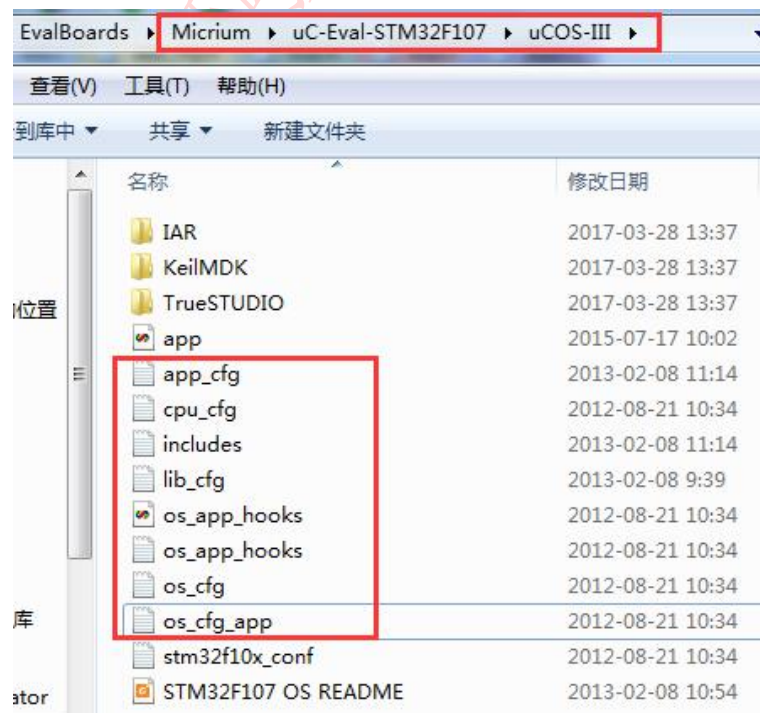
(3) 在 UCOSIII 文件夹内新建 5 个文件夹，如图所示：

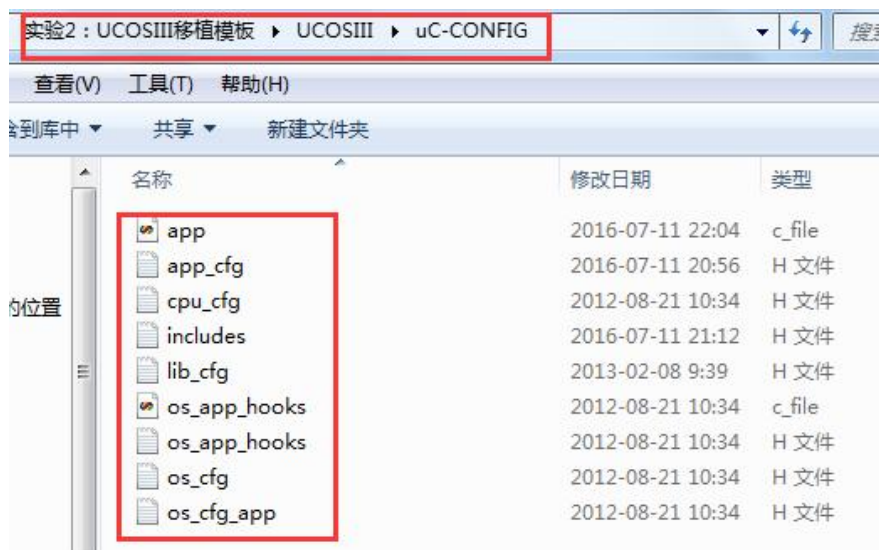


(4) 下面就开始文件的拷贝，将前面下载的官方 UCOSIII 源码目录内的文件拷贝到我们新建的“uC-BSP”文件夹内，如图所示：



(5) 将前面下载的官方 UCOSIII 源码目录内的文件拷贝到我们新建的“uC-CONFIG”文件夹内，如图所示：

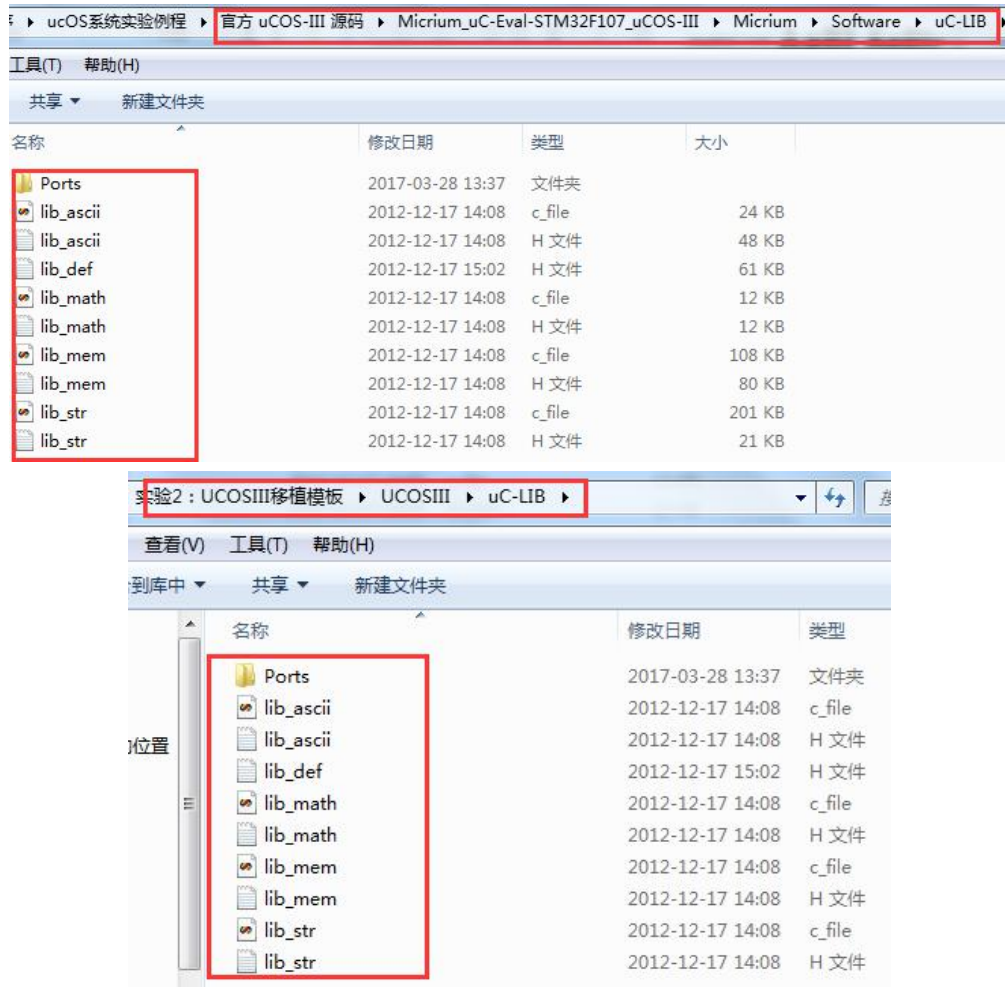




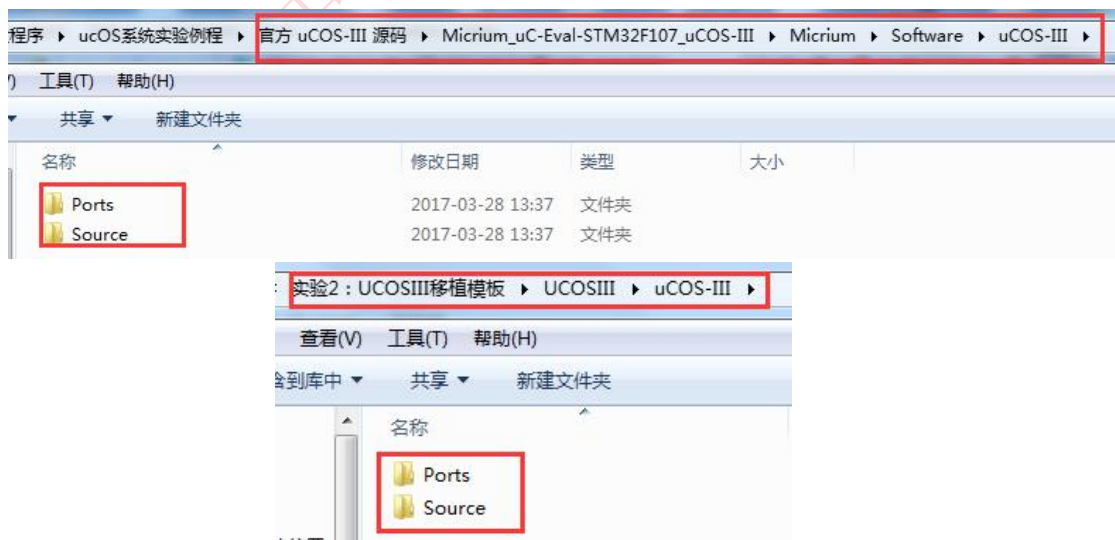
(6) 将前面下载的官方 UCOSIII 源码目录内的文件拷贝到我们新建的“uC-CPU”文件夹内，如图所示：



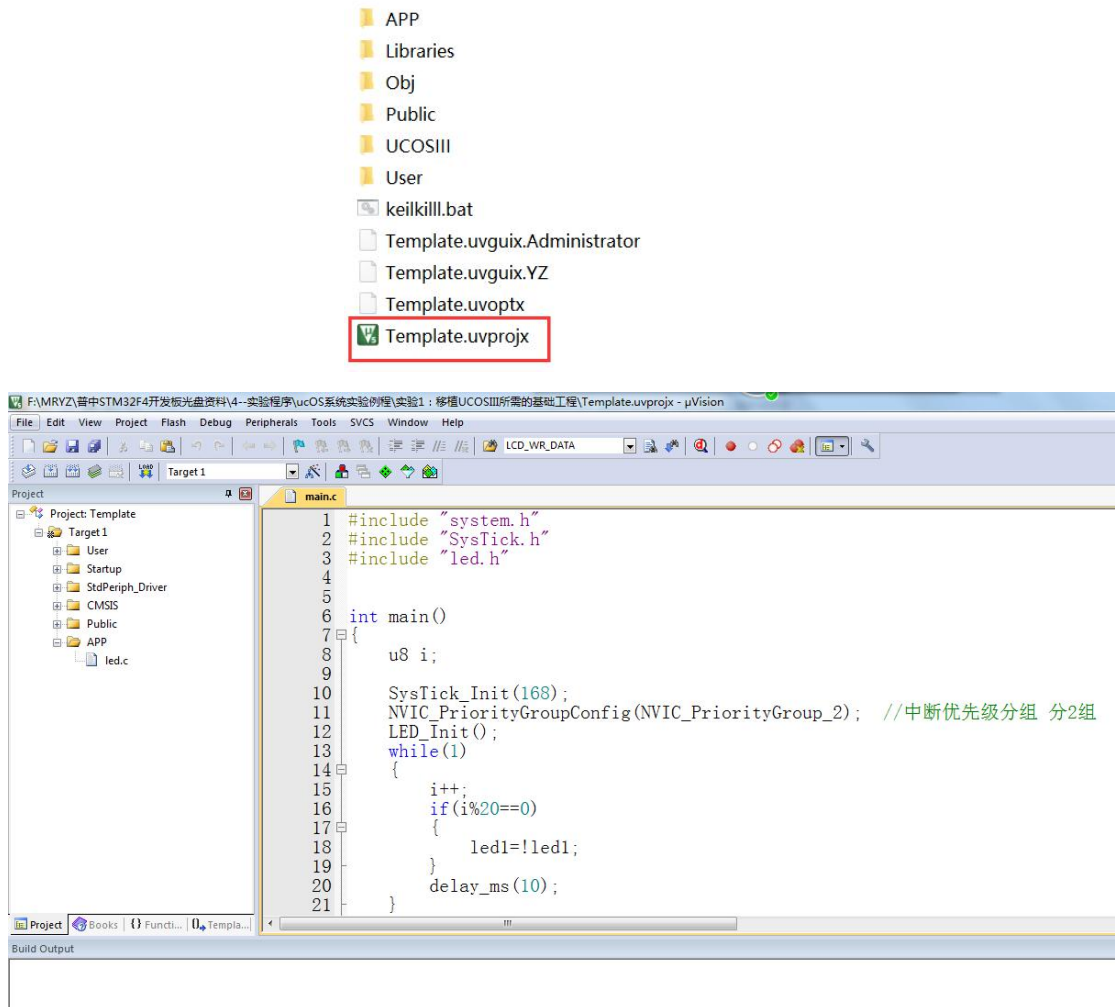
(7) 同理将前面下载的官方 UCOSIII 源码目录内的文件拷贝到我们新建的“uC-LIB”文件夹内，如图所示：



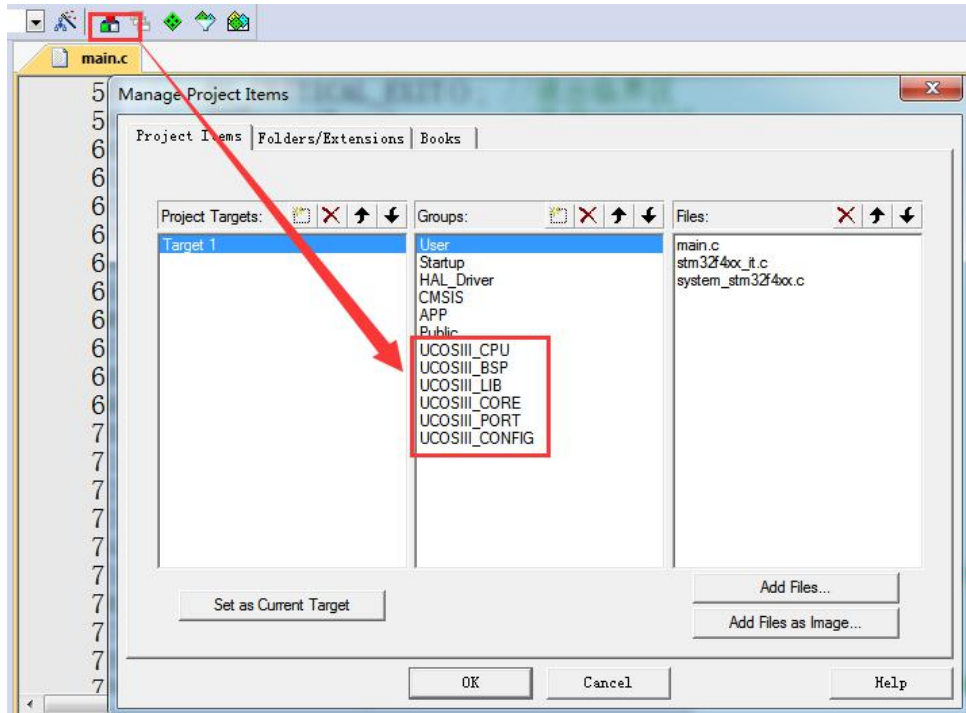
(8) 同理将前面下载的官方 UCOSIII 源码目录内的文件拷贝到我们新建的“uCOS-III”文件夹内，如图所示：



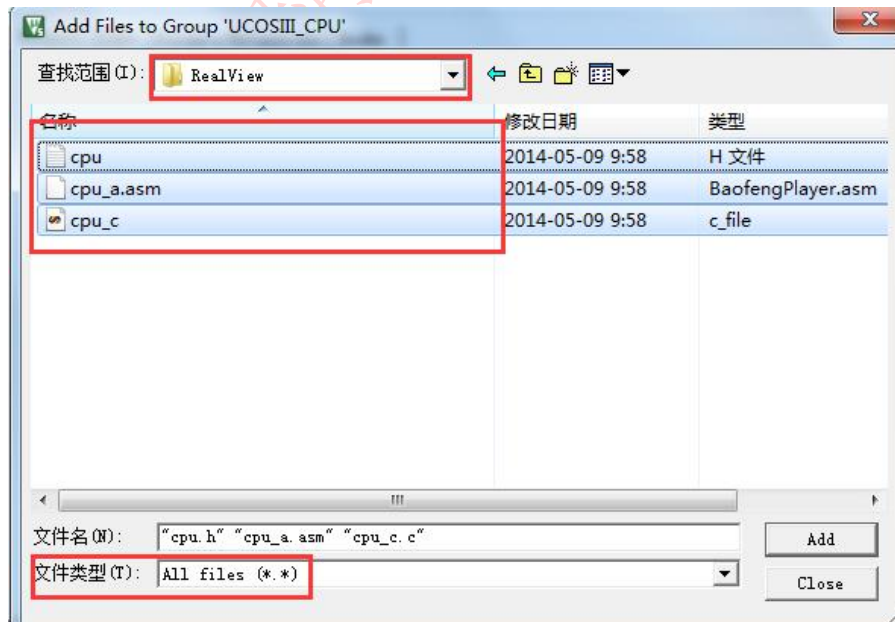
(9) UCOSIII 文件拷贝完成后，接下来我们打开工程，如图所示：

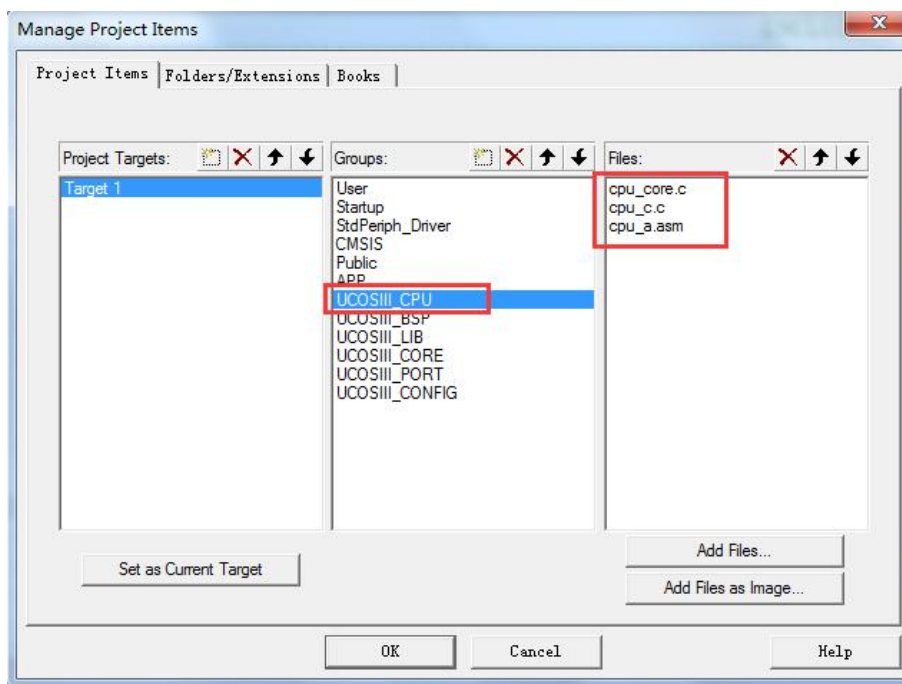


(10) 给工程添加如下工程组（添加工程组在库函数实验中已经讲解过，这里不多说），如图所示：

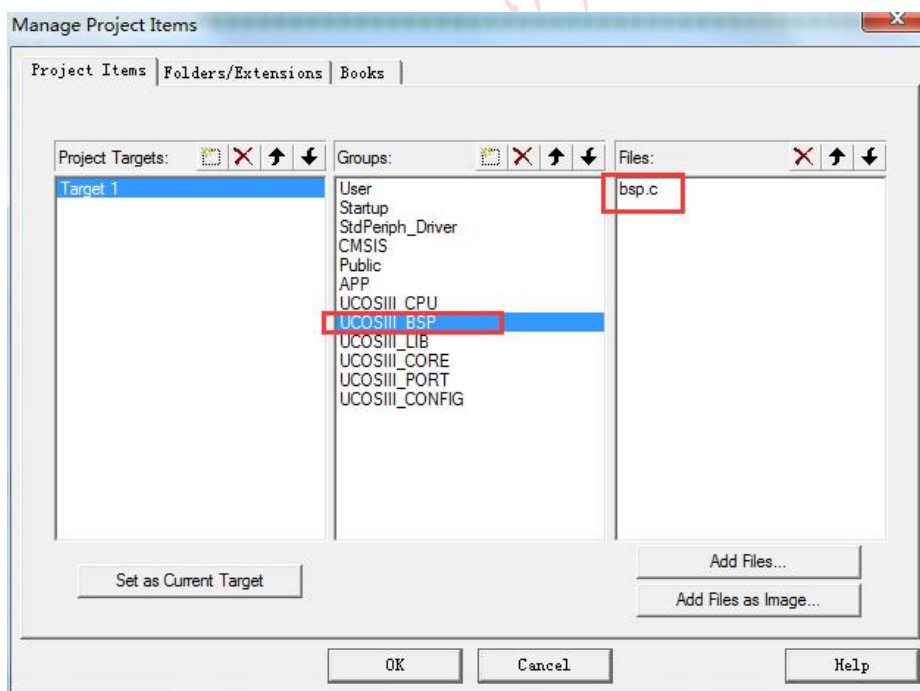


(11) 向“UCOSIII_CPU”组中添加“UCOSIII 移植模板\UCOSIII\uC-CPU”内的“cpu_core.c”和“UCOSIII 移植模板\UCOSIII\uC-CPU\ARM-Cortex-M3\RealView”内的“cpu_a.asm”、“cpu_c.c”文件。（注意：添加 cpu_a.asm 文件时，文件类型选择“All file (*.*)”）
如图所示：

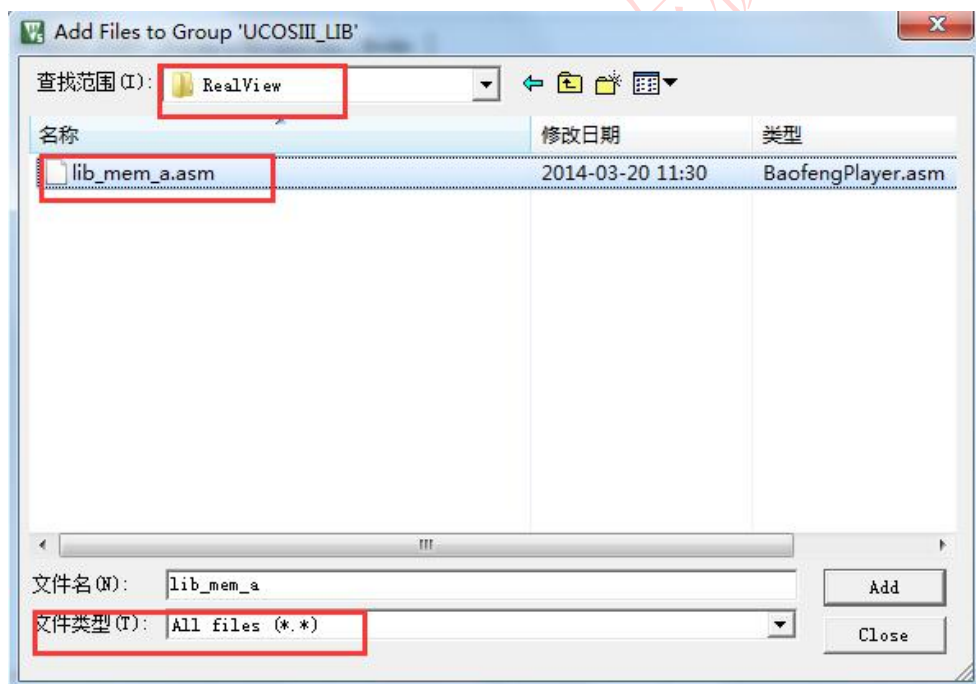
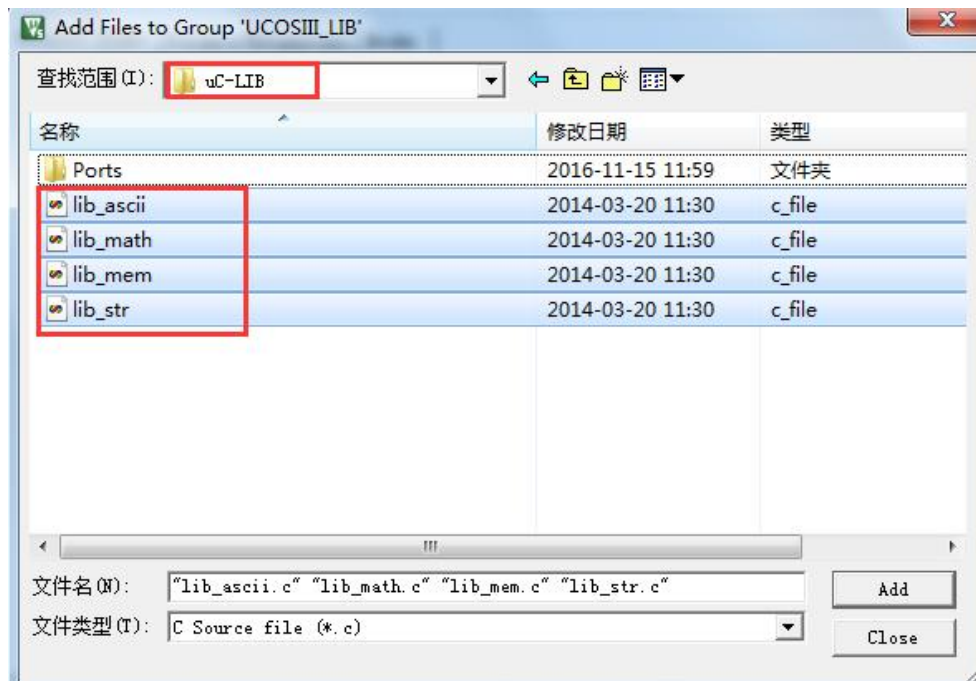


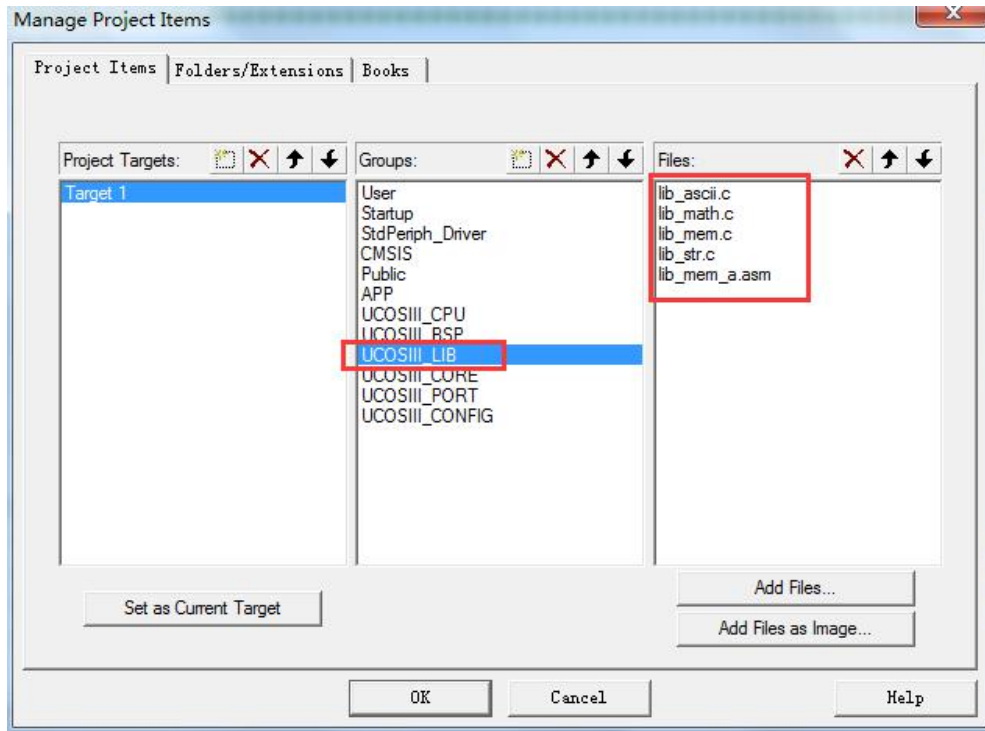


(12) 向“UCOSIII_BSP”组中添加“UCOSIII 移植模板\UCOSIII\uC-BSP”内的“bsp.c”文件。如图所示：

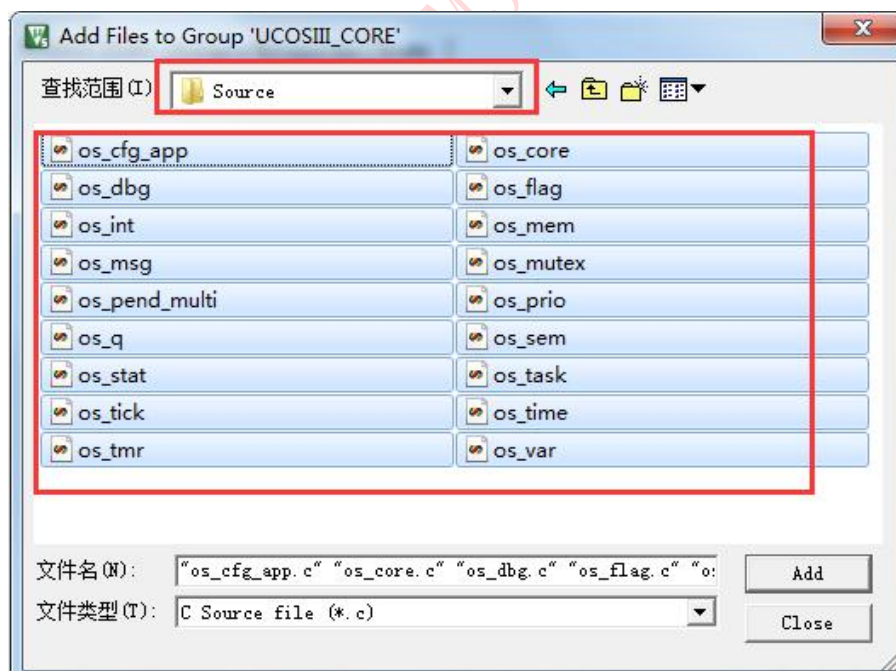


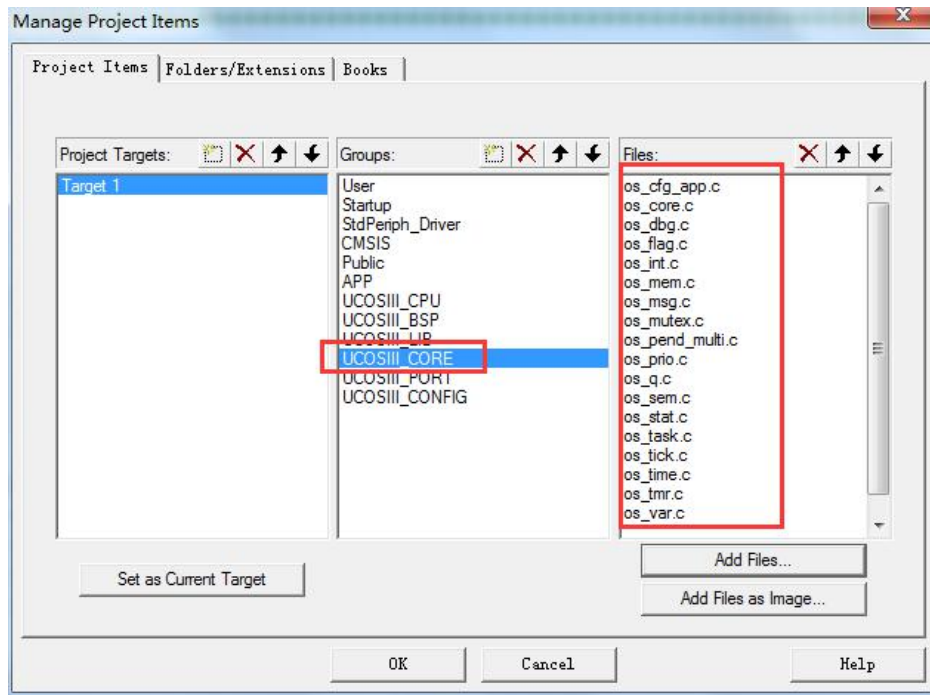
(13) 向“UCOSIII_LIB”组中添加“UCOSIII 移植模板\UCOSIII\uC-LIB”内的所有源文件和“UCOSIII 移植模板\UCOSIII\uC-LIB\Ports\ARM-Cortex-M3\RealView”内的“lib_mem_a.asm”文件。（注意：添加 lib_mem_a.asm 文件时，文件类型选择“**All file (*.*)**”）如图所示：



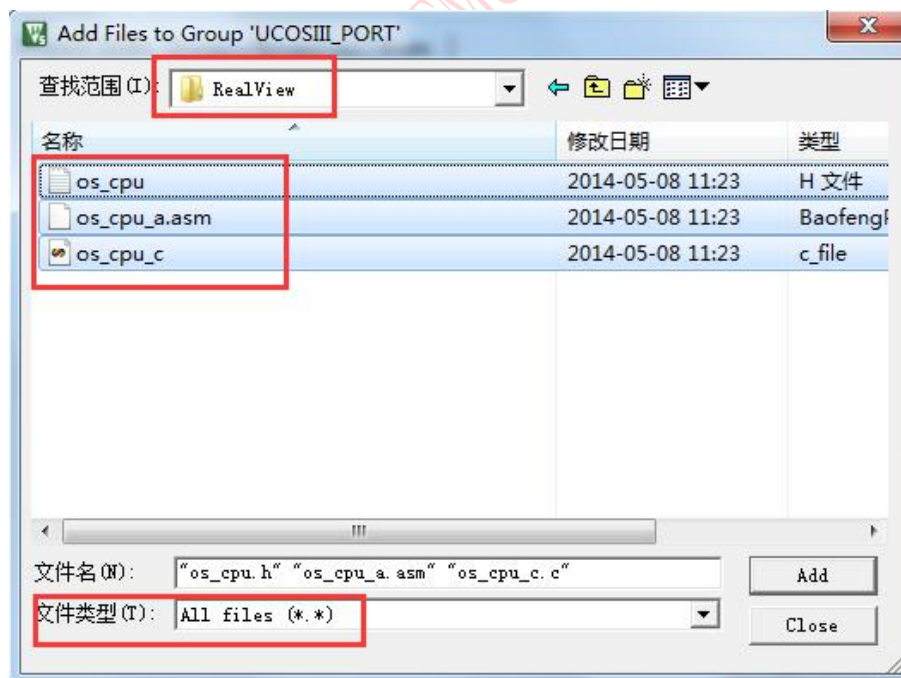


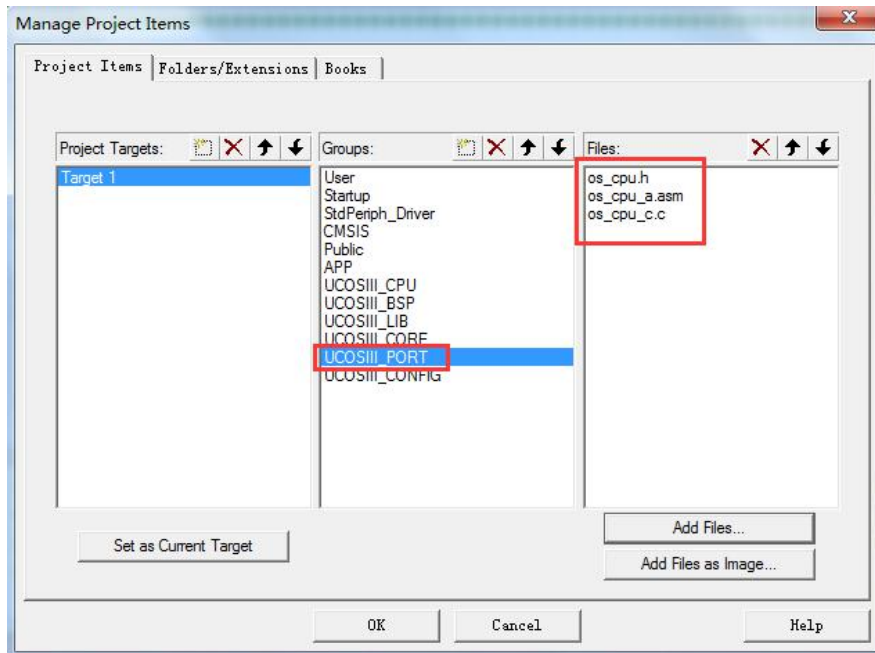
(14) 向“UCOSIII_CORE”组中添加“UCOSIII 移植模板
\\UCOSIII\\uCOS-III\\Source”内的所有源文件。如图所示：



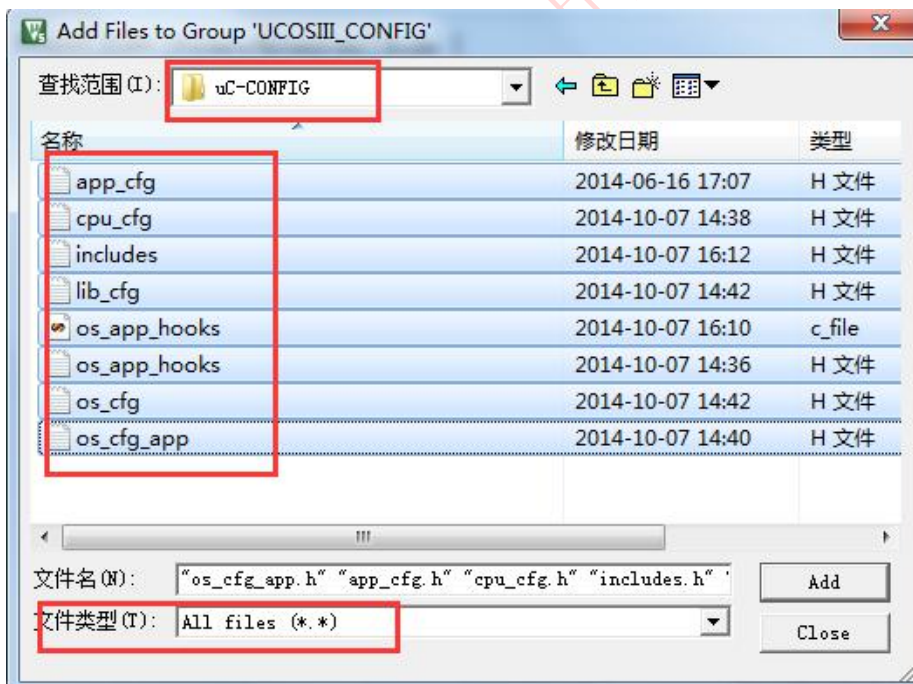


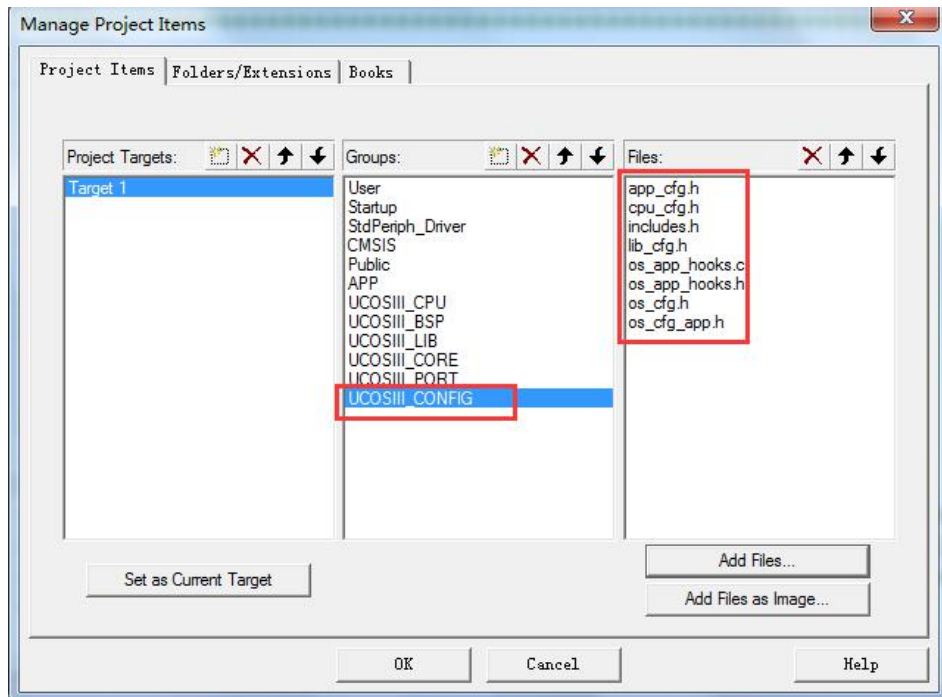
(15) 向“UCOSIII_PORT”组中添加“UCOSIII 移植模板
\\UCOSIII\\uCOS-III\\Ports\\ARM-Cortex-M4\\Generic\\RealView”内的所有文件。
如图所示：



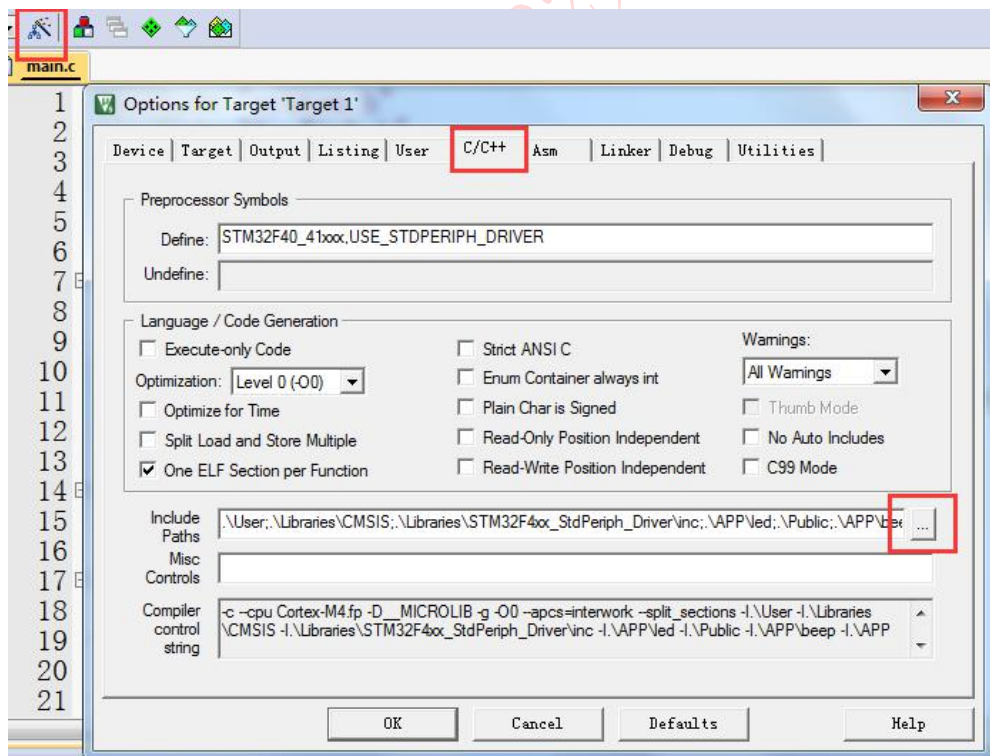


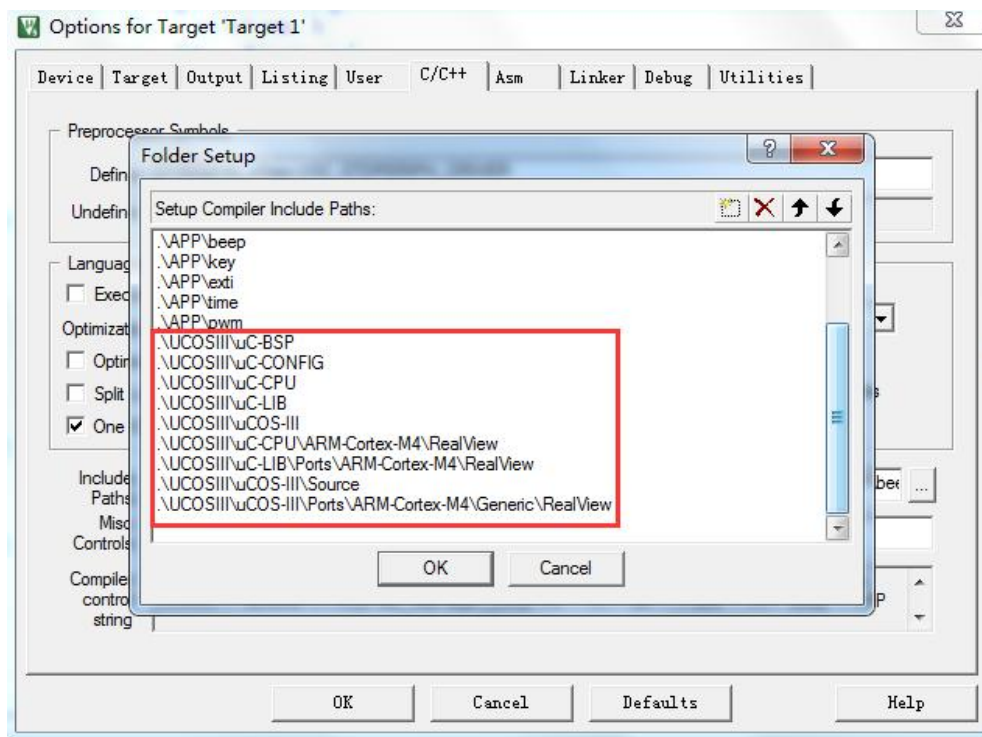
(16) 向“UCOSIII_CONFIG”组中添加“UCOSIII 移植模板
\\UCOSIII\\uC-CONFIG”内的所有文件。如图所示：





(17) 打开魔术棒“Options for Target”，给加进来的工程组添加包含路径。如图所示：





此时可以编译一下整个工程，但肯定会有错误的，UCOSIII 的移植尚未完毕，接下来需要对工程文件进行修改。

1.2.3 修改 bsp.c 和 bsp.h 文件

我们打开 bsp.c 文件，里面有很多的代码我们只需要其中的很少一部分关于 DWT 的代码，因此我们要做相应的修改，在修改之前我们稍微讲解一下 Cortex-M3/M4 的跟踪组件。

在 CM3/CM4 中有 3 种跟踪源：ETM、ITM 和 DWT，要想使用 ETM、ITM 和 DWT 的话要将 DEMCR 寄存器的 TRCENA 位(bit24)置 1，DEMCR 寄存器地址为：0XE000EDFC，在我们的 STM32 参考资料下的《Cortex-M3 与 M4 权威指南》(英文名为《The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd Edition》)的 501 页有详细的讲解，感兴趣的朋友可以自行查阅一下。在 DWT 组件中有一个 CYCCNT 寄存器，这个寄存器用来对时钟周期计数，我们可以使用这个寄存器来测量执行某个任务所花费的时间。

DWT 组件有多个寄存器，我们这里只使用 DWT 的控制寄存器 CTRL、CYCCNT 寄存器，CTRL 寄存器地址为 0XE0001000、CYCCNT 寄存器地址为 0XE0001004。如果我们要使用时钟计数功能需要将 CTRL 寄存器的 bit0 置 1。

这里大家可以直接使用我们移植好的模板“实验 2: UC0SIII 移植模板”实验中的 bsp.c 文件, 修改后的 bsp.c 文件如下, 这里为了方便讲解我们删掉了一些函数的英文注释, 改为中文注释, 在我们提供的例程中并没有添加这些函数的中文注释。

```

#define BSP_MODULE
#include <bsp.h>

#define BSP_REG_DEM_CR    (*(CPU_REG32 *)0xE000EDFC) //DEMCR 寄存器
#define BSP_REG_DWT_CR    (*(CPU_REG32 *)0xE0001000) //DWT 控制寄存器
#define BSP_REG_DWT_CYCCNT (*(CPU_REG32 *)0xE0001004) //DWT 时钟计数寄存器
#define BSP_REG_DBGMCU_CR  (*(CPU_REG32 *)0xE0042004)

//DEMCR 寄存器的第 24 位, 如果要使用 DWT ETM ITM 和 TPIU 的话 DEMCR 寄存器的第 24 位置 1
#define BSP_BIT_DEM_CR_TRCENA    DEF_BIT_24
//DWTCR 寄存器的第 0 位, 当为 1 的时候使能 CYCCNT 计数器, 使用 CYCCNT 之前应当先初始化
#define BSP_BIT_DWT_CR_CYCCNTENA    DEF_BIT_00

//获取系统时钟
//返回值: 系统时钟 HCLK
CPU_INT32U BSP_CPU_ClkFreq (void)
{
    RCC_ClocksTypeDef  rcc_clocks;

    RCC_GetClocksFreq(&rcc_clocks);    //获取各个时钟频率

    return ((CPU_INT32U)rcc_clocks.HCLK_Frequency); //返回 HCLK 时钟频率
}

//此函数用来开启和初始化 CPU 的时间戳定时器, 其实就是使能 DWT 和 CYCCNT, 并且初始
//化 CYCCNT 为 0。
#if (CPU_CFG_TS_TMR_EN == DEF_ENABLED)
void CPU_TS_TmrInit (void)
{
    CPU_INT32U  fclk_freq;

```

```

fclk_freq = BSP_CPU_ClkFreq();

BSP_REG_DEM_CR      |= (CPU_INT32U)BSP_BIT_DEM_CR_TRCENA;//使能
DWT
BSP_REG_DWT_CYCCNT  = (CPU_INT32U)0u;//初始化 CYCCNT 寄存器
BSP_REG_DWT_CR      |= (CPU_INT32U)BSP_BIT_DWT_CR_CYCCNTENA;//
开启 CYCCNT

CPU_TS_TmrFreqSet((CPU_TS_TMR_FREQ)fclk_freq);
}
#endif
//此函数其实就是获取 CYCCNT 中的值
#if (CPU_CFG_TS_TMR_EN == DEF_ENABLED)
CPU_TS_TMR CPU_TS_TmrRd (void)
{
    CPU_TS_TMR  ts_tmr_cnts;

    ts_tmr_cnts = (CPU_TS_TMR)BSP_REG_DWT_CYCCNT;

    return (ts_tmr_cnts);
}
#endif
//下面这两个函数：CPU_TS32_to_uSec()和 CPU_TS64_to_uSec()是用来将
读取到的时钟周期数
//转换为 us。
#if (CPU_CFG_TS_32_EN == DEF_ENABLED)
CPU_INT64U CPU_TS32_to_uSec (CPU_TS32  ts_cnts)
{
    CPU_INT64U  ts_us;
    CPU_INT64U  fclk_freq;

    fclk_freq = BSP_CPU_ClkFreq();
    ts_us      = ts_cnts / (fclk_freq / DEF_TIME_NBR_uS_PER_SEC);

    return (ts_us);
}
#endif

#if (CPU_CFG_TS_64_EN == DEF_ENABLED)
CPU_INT64U CPU_TS64_to_uSec (CPU_TS64  ts_cnts)
{

```

```
CPU_INT64U  ts_us;
CPU_INT64U  fclk_freq;

fclk_freq = BSP_CPU_ClkFreq();
ts_us      = ts_cnts / (fclk_freq / DEF_TIME_NBR_uS_PER_SEC);

return (ts_us);
}
#endif
```

接下来我们还要对 bsp.h 文件做相应的修改,修改后的 bsp.h 的文件如下所示, bsp.h 文件很简单,就是添加一些头文件。

```
#ifndef  BSP_PRESENT
#define  BSP_PRESENT

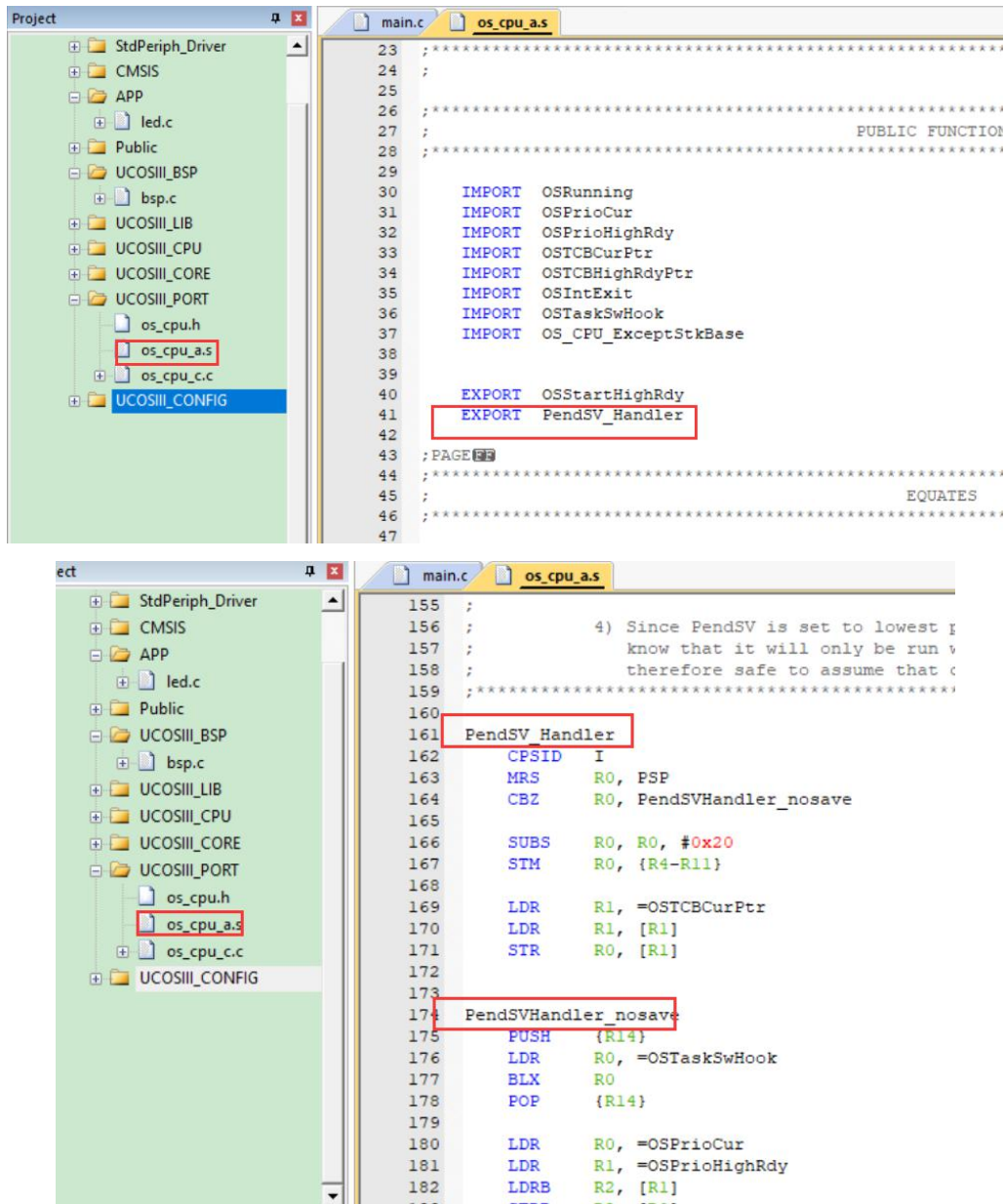
#ifdef   BSP_MODULE
#define  BSP_EXT
#else
#define  BSP_EXT  extern
#endif

#include <stdio.h>
#include <stdarg.h>
#include <cpu.h>
#include <cpu_core.h>
#include <lib_def.h>
#include <lib_ascii.h>
#include <stm32f10x_conf.h>

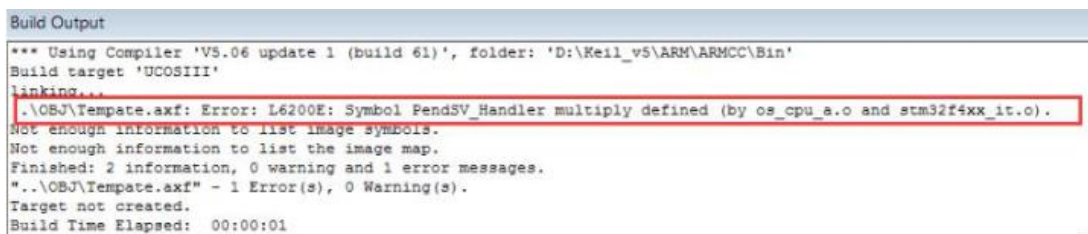
#endif
```

1.2.4 修改 os_cpu_a.asm 文件

Micrium 官方的移植工程修改了 PendSV 中断服务函数的名字,改为了 OS_CPU_PendSVHandler,我们需要将 os_cpu_a.asm 文件中的所有 OS_CPU_PendSVHandler 改为 PendSV_Handler,如下所示:

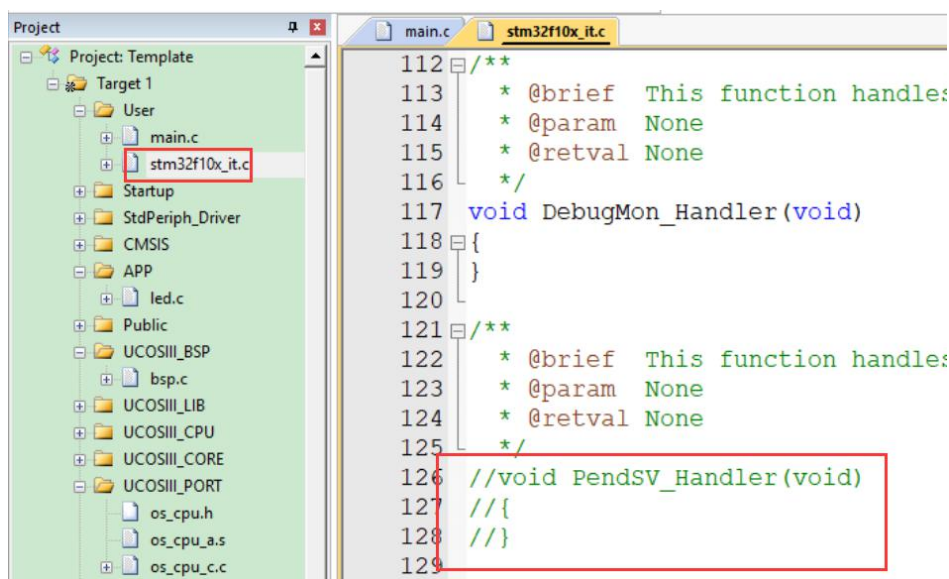


修改完成以后编译会有如下图所示错误提示，提示我们 PendSV_Handler() 函数重复定义。



这是因为在 stm32f10x_it.c 文件中 ST 官方已经给出了函数 PendSV_Handler(), 而 os_cpu_a.asm 又写了一个 PnedSV_Handler(), 造成重复定义的错误, 因此肯定要消除掉一个。我们将 stm32f10x_it.c 中的函数

PendSV_Handler() 屏蔽掉就可以了，如下图所示：



1.2.5 修改 os_cfg_app.h 文件

我们还需要修改 os_cfg_app.h 文件，os_cfg_app.h 主要是对 UCOSIII 内部一些系统任务的配置，如任务优先级、任务堆栈、UCOSIII 的系统时钟节拍等等，os_cfg_app.h 文件都是一些宏定义，比较简单，该文件在“UCOSIII_CONFIG”下可找到，文件代码如下。

```
#ifndef OS_CFG_APP_H
#define OS_CFG_APP_H

#define OS_CFG_MSG_POOL_SIZE          100u    //消息数量

#define OS_CFG_ISR_STK_SIZE            100u    //中断任务堆栈大小
//任务堆栈深度，比如当定义为 10 时表示当任务堆栈剩余百分之 10%时就
说明堆栈为空
#define OS_CFG_TASK_STK_LIMIT_PCT_EMPTY 10u

//*****空闲任务*****
#define OS_CFG_IDLE_TASK_STK_SIZE      128u    //空任务堆栈大小

//*****中断服务管理任务*****
#define OS_CFG_INT_Q_SIZE               10u    //中断队列大小
#define OS_CFG_INT_Q_TASK_STK_SIZE     100u    //中断服务管理任务
大小

//*****统计任务*****
```

```

//统计任务优先级，倒数第二个优先级，最后一个优先级是空闲任务的
#define OS_CFG_STAT_TASK_PRIO      (OS_CFG_PRIO_MAX - 2u)
// OS_CFG_STAT_TASK_RATE_HZ 用于统计任务计算 CPU 使用率，统计任务
通过统计在
// (1/ OS_CFG_STAT_TASK_RATE_HZ) 秒的时间内 OSStatTaskCtr 能够达
到的最大值来得到
//CPU 使用率，OS_CFG_STAT_TASK_RATE_HZ 值应该在 1~10Hz
#define OS_CFG_STAT_TASK_RATE_HZ    10u
#define OS_CFG_STAT_TASK_STK_SIZE    100u //统计任务堆栈

//*****时钟节拍任务*****
#define OS_CFG_TICK_RATE_HZ          1000u //系统时钟节拍频率
//时钟节拍任务，一般设置一个相对较高的优先级
#define OS_CFG_TICK_TASK_PRIO        1u
#define OS_CFG_TICK_TASK_STK_SIZE    128u //时钟节拍任务堆栈
大小

//*****定时任务*****
#define OS_CFG_TMR_TASK_PRIO          2u //定时任务优先级
#define OS_CFG_TMR_TASK_RATE_HZ      100u //定时频率，一般为
100Hz
#define OS_CFG_TMR_TASK_STK_SIZE      128u //定时任务堆栈大小

#endif

```

在 UCOSIII 中有五个系统任务：空闲任务、时钟节拍任务、统计任务、定时任务和中断服务管理任务，在系统初始化的时候至少要创建两个任务：空闲任务和时钟节拍任务。空闲任务的优先级应该为最低 $OS_CFG_PRIO_MAX-1$ ，如果使用中断服务管理任务的话那么中断服务管理任务的优先级应该为最高 0。其他 3 个任务的优先级用户可以自行设置，本教程中我们将统计任务设置为 $OS_CFG_PRIO_MAX-2$ ，即倒数第二个优先级；时钟节拍任务也需要一个高优先级，我们将优先级 1 分配给时钟节拍任务；将优先级 2 分配给定时器任务。所以优先级 0、1、2、 $OS_CFG_PRIO_MAX-2$ 和 $OS_CFG_PRIO_MAX-1$ 这 5 个优先级用户应用程序是不能使用的。

此时 UCOSIII 的移植尚未完毕，接下来还需要对工程文件进行修改。

1.2.6 修改 Public 文件夹内文件

(1) 修改 SysTick.c 文件

为了能在 UCOS 和 delay 中能共用 systick 定时器，这里介绍一种方法。

首先，我们简单介绍下 UCOSIII 的时钟：ucos 运行需要一个系统时钟节拍（类似“心跳”），而这个节拍是固定的（由 OS_TICKS_PER_SEC 宏定义设置），比如要求 5ms 一次（即可设置：OS_TICKS_PER_SEC=200），在 STM32 上面，一般是由 SysTick 来提供这个节拍，也就是 SysTick 要设置为 5ms 中断一次，为 ucos 提供时钟节拍，而且这个时钟一般是不能被打断的（否则就不准了）

因为在 ucos 下 systick 不能再被随意更改，如果我们还想利用 systick 来做 delay_us 或者 delay_ms 的延时，就必须想点办法了，这里我们利用的是时钟摘取法。以 delay_us 为例，比如 delay_us(50)，在刚进入 delay_us 的时候先计算好这段延时需要等待的 systick 计数次数，这里为 50*72（假设系统时钟为 72Mhz，因为我们设置 systick 的频率为系统时钟频率，那么 systick 每增加 1，就是 1/72us），然后我们就一直统计 systick 的计数变化，直到这个值变化了 50*72，一旦检测到变化达到或者超过这个值，就说明延时 50us 时间到了。这样，我们只是抓取 SysTick 计数器的变化，并不需要修改 SysTick 的任何状态，完全不影响 SysTick 作为 UCOS 时钟节拍的功能，这就是实现 delay 和操作系统共用 SysTick 定时器的原理。

①新增操作系统支持宏及相关函数

当需要 delay_ms 和 delay_us 支持操作系统的时候，我们需要用到 3 个宏定义和 4 个函数，宏定义及函数代码如下：

```
#include "includes.h"//添加 UCOS 头文件

//支持 UCOSII
#ifdef OS_CRITICAL_METHOD
//OS_CRITICAL_METHOD 定义了，说明要支持 UCOSII
#define delay_osrunning OSRunning //OS 是否运行标记, 0, 不运行;1, 在运行
#define delay_ostickspersec OS_TICKS_PER_SEC //OS 时钟节拍，即每秒调度次数
#define delay_osintnesting OSIntNesting //中断嵌套级别，即中断嵌套次数
#endif

//支持 UCOSIII
#ifdef CPU_CFG_CRITICAL_METHOD
//CPU_CFG_CRITICAL_METHOD 定义了，说明要支持 UCOSIII
```

```

#define delay_osrunning      OSRunning          //OS 是否运行标记, 0,
不运行;1, 在运行
#define delay_ostickspersec  OSCfg_TickRate_Hz //OS 时钟节拍, 即每秒
调度次数
#define delay_osintnesting   OSIntNestingCtr    //中断嵌套级别,
即中断嵌套次数
#endif

//us 级延时, 关闭任务调度(防止打断 us 级延迟)
void delay_osschedlock(void)
{
#ifdef CPU_CFG_CRITICAL_METHOD          //使用 UCOSIII
    OS_ERR err;
    OSSchedLock(&err);                  //UCOSIII 的方式, 禁止调
度, 防止打断 us 延时
#else                                    //否则 UCOSII
    OSSchedLock();                      //UCOSII 的方式, 禁止调
度, 防止打断 us 延时
#endif
}

//us 级延时, 恢复任务调度
void delay_osschedunlock(void)
{
#ifdef CPU_CFG_CRITICAL_METHOD          //使用 UCOSIII
    OS_ERR err;
    OSSchedUnlock(&err);                //UCOSIII 的方式, 恢复调度
#else                                    //否则 UCOSII
    OSSchedUnlock();                    //UCOSII 的方式, 恢复调度
#endif
}

//调用 OS 自带的延时函数延时
//ticks:延时的节拍数
void delay_ostimedly(u32 ticks)
{
#ifdef CPU_CFG_CRITICAL_METHOD
    OS_ERR err;
    OSTimeDly(ticks, OS_OPT_TIME_PERIODIC, &err); //UCOSIII 延时采用
周期模式
#else
    OSTimeDly(ticks);                    //UCOSII 延时
#endif
}

```

```
}

//systick 中断服务函数, 使用 OS 时用到
void SysTick_Handler(void)
{
    if(delay_osrunning==1)                //OS 开始跑了, 才执行正常的调度处理
    {
        OSIntEnter();                    //进入中断
        OSTimeTick();                    //调用 ucOS 的时钟服务程序
        OSIntExit();                      //触发任务切换软中断
    }
}
```

支持 OS 需要用到的三个宏定义（以 UCOSII 为例）即：

```
#define delay_osrunning OSRunning //OS 是否运行标记, 0, 不运行; 1, 在运行
#define delay_ostickspersec OS_TICKS_PER_SEC //OS 时钟节拍, 即每秒调度次数
#define delay_osintnesting OSIntNesting //中断嵌套级别, 即中断嵌套次数
```

宏定义：delay_osrunning，用于标记 OS 是否正在运行，当 OS 已经开始运行时，该宏定义值为 1，当 OS 还未运行时，该宏定义值为 0。

宏定义：delay_ostickspersec，用于表示 OS 的时钟节拍，即 OS 每秒钟任务调度次数。

宏定义：delay_osintnesting，用于表示 OS 中断嵌套级别，即中断嵌套次数，每进入一个中断，该值加 1，每退出一个中断，该值减 1。

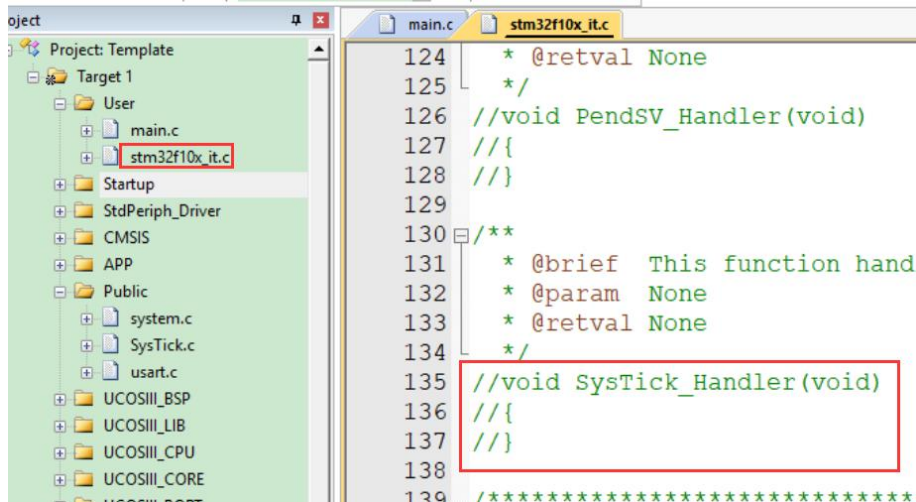
支持 OS 需要用到的 4 个函数，即：

函数：delay_osschedlock，用于 delay_us 延时，作用是禁止 OS 进行调度，以防打断 us 级延时，导致延时时间不准。

函数：delay_osschedunlock，同样用于 delay_us 延时，作用是在延时结束后恢复 OS 的调度，继续正常的 OS 任务调度。

函数：delay_ostimedly，则是调用 OS 自带的延时函数，实现延时。该函数的参数为时钟节拍数。

函数：SysTick_Handler，则是 systick 的中断服务函数，该函数为 OS 提供时钟节拍，同时可以引起任务调度。由于在 stm32f10x_it.c 中也定义了一个 SysTick_Handler 函数，因此需要把 stm32f10x_it.c 中的注释掉。如下所示：



以上就是 delay_ms 和 delay_us 支持操作系统时，需要实现的 3 个宏定义和 4 个函数。

②修改 SysTick_Init 函数

该函数用来初始化 2 个重要参数：fac_us 以及 fac_ms；同时把 SysTick 的时钟源选择为外部时钟，该函数会根据 delay_ostickspersec 宏的设置来配置 SysTick 的中断时间，并开启 SysTick 中断。修改后代码如下：

```
//初始化延迟函数
//SYSTICK的时钟固定为AHB时钟的1/8
//SYSCLK:系统时钟频率
void SysTick_Init(u8 SYSCLK)
{
    u32 reload;
    SysTick_CLKSourceConfig(SysTick_CLKSource_HCLK_Div8);
    fac_us=SYSCLK/8;           //不论是否使用OS, fac_us 都需要使用
    reload=SYSCLK/8;           //每秒钟的计数次数
    reload*=1000000/delay_ostickspersec; //根据delay_ostickspersec 设定溢出时间
    //reload 为 24 位寄存器,
    //最大值:16777216, 在 72M 下, 约合 1.86s 左右
    fac_ms=1000/delay_ostickspersec; //代表 OS 可以延时的最少单位

    SysTick->CTRL|=SysTick_CTRL_TICKINT_Msk; //开启 SYSTICK 中断
    SysTick->LOAD=reload; //每1/delay_ostickspersec 秒中断一次
    SysTick->CTRL|=SysTick_CTRL_ENABLE_Msk; //开启 SYSTICK
```

```
}
```

③修改 delay_us 函数

该函数用来延时指定的 us，其参数 nus 为要延时的微秒数。修改后代码如下：

```
//延时 nus
//nus 为要延时的 us 数.
void delay_us(u32 nus)
{
    u32 ticks;
    u32 told, tnow, tcnt=0;
    u32 reload=SysTick->LOAD;           //LOAD 的值

    ticks=nus*fac_us;                   //需要的节拍数

    tcnt=0;
    delay_osschedlock();                //阻止 OS 调度，防止打断
us 延时
    told=SysTick->VAL;                  //刚进入时的计数器值
    while(1)
    {
        tnow=SysTick->VAL;
        if(tnow!=told)
        {
            if(tnow<told)tcnt+=told-tnow;    // 这里注意一下
SYSTICK 是一个递减的计数器就可以了.
            else tcnt+=reload-tnow+told;
            told=tnow;
            if(tcnt>=ticks)break;           //时间超过/等于要延
迟的时间, 则退出.
        }
    };
    delay_osschedunlock();              //恢复 OS 调度
}
```

这里就正是利用了我们前面提到的时钟摘取法，ticks 是延时 nus 需要等待的 SysTick 计数次数（也就是延时时间），told 用于记录最近一次的 SysTick->VAL 值，然后 tnow 则是当前的 SysTick->VAL 值，通过他们的对比累加，实现 SysTick 计数次数的统计，统计值存放在 tcnt 里面，然后通过对比 tcnt 和 ticks，来判断延时是否到达，从而达到不修改 SysTick 实现 nus 的延时。使用 delay_osschedlock 和 delay_osschedunlock 两个函数，用于调

度上锁和解锁，这是为了防止 OS 在 delay_us 的时候打断延时，可能导致的延时不准，所以我们利用这两个函数来实现免打断，从而保证延时精度

④修改 delay_ms 函数

该函数是用来延时指定的 ms 的，其参数 nms 为要延时的毫秒数。修改后代码如下：

```
//延时 nms
//nms:要延时的 ms 数
void delay_ms(u16 nms)
{
    if(delay_osrunning&&delay_osintnesting==0)    //如果 OS 已经在跑了,并且不是在中断里面(中断里面不能任务调度)
    {
        if(nms>=fac_ms)                            //延时的时间大于 OS
        的最少时间周期
        {
            delay_ostimedly(nms/fac_ms);            //OS 延时
        }
        nms%=fac_ms;                                //OS 已经无法提供这么小
        的延时了,采用普通方式延时
    }
    delay_us((u32)(nms*1000));                      //普通方式延时
}
```

该函数中 delay_osrunning 是 OS 正在运行的标志，delay_osintnesting 则是 OS 中断嵌套次数，必须 delay_osrunning 为真，且 delay_osintnesting 为 0 的时候，才可以调用 OS 自带的延时函数进行延时（可以进行任务调度），delay_ostimedly 函数就是利用 OS 自带的延时函数，实现任务级延时的，其参数代表延时的时钟节拍数（假设 delay_ostickspersec=200，那么 delay_ostimedly (1)，就代表延时 5ms）。

当 OS 还未运行的时候，我们的 delay_ms 就是直接由 delay_us 实现的，OS 下的 delay_us 可以实现很长的延时（达到 204 秒）而不溢出！，所以放心的使用 delay_us 来实现 delay_ms，不过由于 delay_us 的时候，任务调度被上锁了，所以还是建议不要用 delay_us 来延时很长的时间，否则影响整个系统的性能。

当 OS 运行的时候，我们的 delay_ms 函数将先判断延时时长是否大于等于 1 个 OS 时钟节拍（fac_ms），当大于这个值的时候，我们就通过调用 OS 的延

时函数来实现（此时任务可以调度），不足 1 个时钟节拍的时候，直接调用 delay_us 函数实现（此时任务无法调度）

（2）修改 usart.c/usart.h 文件

要使用 UCOS，同样要添加#include "includes.h"头文件，并且为了不让操作系统线程干扰到串口中断的运行，所以通常需要在进入中断时加入 OSIntEnter()函数，退出中断前加上 OSIntExit()函数。如下：

```
void USART1_IRQHandler(void) //串口 1 中断服务程序
{
    u8 r;
    OSIntEnter();
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //接收中
断
    {
        r =USART_ReceiveData(USART1); //(USART1->DR); //读取接收到的
数据
        if((USART1_RX_STA&0x8000)==0)//接收未完成
        {
            if(USART1_RX_STA&0x4000)//接收到了 0x0d
            {
                if(r!=0x0a)USART1_RX_STA=0;//接收错误,重新开始
                else USART1_RX_STA|=0x8000; //接收完成了
            }
            else //还没收到 0x0d
            {
                if(r==0x0d)USART1_RX_STA|=0x4000;
                else
                {
                    USART1_RX_BUF[USART1_RX_STA&0X3FFF]=r;
                    USART1_RX_STA++;

                    if(USART1_RX_STA>(USART1_REC_LEN-1))USART1_RX_STA=0;//接收数据错
误,重新开始接收
                }
            }
        }
    }
    OSIntExit();
}
```

（3）修改 system.c/system.h 文件

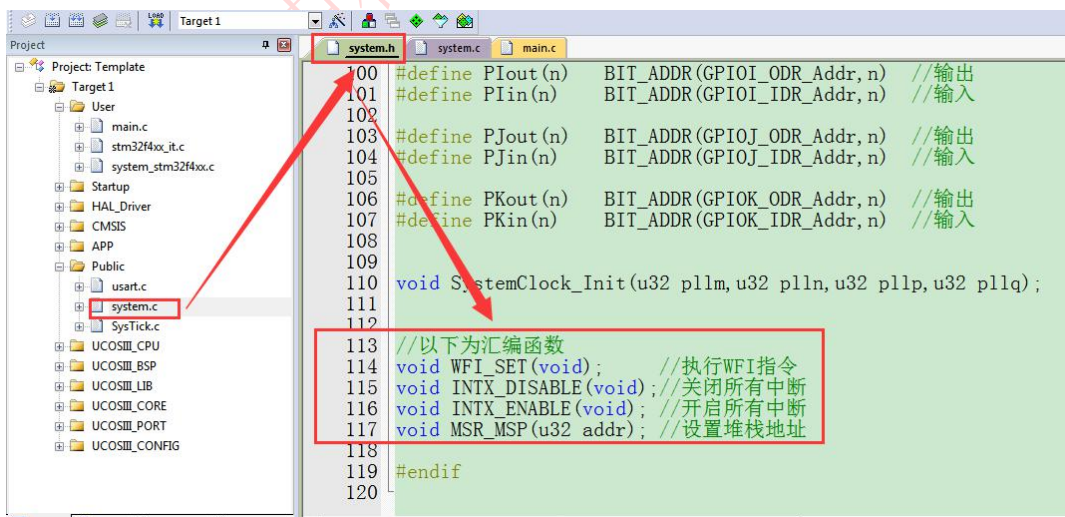
在 system.c 文件内代码最后加上如下汇编代码：

```

//THUMB 指令不支持汇编内联
//采用如下方法实现执行汇编指令 WFI
__asm void WFI_SET(void)
{
    WFI;
}
//关闭所有中断(但是不包括 fault 和 NMI 中断)
__asm void INTX_DISABLE(void)
{
    CPSID    I
    BX       LR
}
//开启所有中断
__asm void INTX_ENABLE(void)
{
    CPSIE    I
    BX       LR
}
//设置栈顶地址
//addr:栈顶地址
__asm void MSR_MSP(u32 addr)
{
    MSR MSP, r0          //set Main Stack value
    BX r14
}

```

同时在 system.h 文件内将新增的几个汇编函数声明，如下：



到这里 UCOSIII 的移植已经完成，大家可以编译下工程，没有错误和警告（如果出现错误，请按照对应的错误提示修改）。接下来就来验证 UCOSIII 系统的运行情况。

在“main.c”文件内创建 2 个任务，一个起始任务，在起始任务中创建一个 LED 任务，用来切换 D1 指示灯状态。代码如下：

```
#include "system.h"
#include "SysTick.h"
#include "usart.h"
#include "led.h"
#include "includes.h"

//任务优先级
#define START_TASK_PRIO    3
//任务堆栈大小
#define START_STK_SIZE     512
//任务控制块
OS_TCB StartTaskTCB;
//任务堆栈
CPU_STK START_TASK_STK[START_STK_SIZE];
//任务函数
void start_task(void *p_arg);

//任务优先级
#define LED1_TASK_PRIO     4
//任务堆栈大小
#define LED1_STK_SIZE     128
//任务控制块
OS_TCB Led1TaskTCB;
//任务堆栈
CPU_STK LED1_TASK_STK[LED1_STK_SIZE];
void led1_task(void *p_arg);

int main(void)
{
    OS_ERR err;
    CPU_SR_ALLOC();

    SysTick_Init(72);
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级
    分组 分 2 组
    LED_Init();
    USART1_Init(115200);
```

```

OSInit(&err);    //初始化 UCOSIII
OS_CRITICAL_ENTER(); //进入临界区
//创建开始任务
OSTaskCreate((OS_TCB * )&StartTaskTCB,    //任务控制块
              (CPU_CHAR * )"start task",    //任务名字
              (OS_TASK_PTR )start_task,    //任务函数
              (void * )0,                  //传递给任务函数
              (OS_PRIO )START_TASK_PRIO,    //任务优先级
              (CPU_STK * )&START_TASK_STK[0], //任务堆栈基
              (CPU_STK_SIZE)START_TASK_SIZE/10, //任务堆栈深
              (CPU_STK_SIZE)START_TASK_SIZE,    //任务堆栈大
              (OS_MSG_QTY )0,                  //任务内部消息队
              (OS_TICK )0,                    //当使能时间片轮
              (void * )0,                    //用户补充的存储
              (OS_OPT )OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR|OS_OPT_TASK
              _SAVE_FP, //任务选项,为了保险起见,所有任务都保存浮点寄存器的值
              (OS_ERR * )&err);              //存放该函数错误
//开始任务函数
void start_task(void *p_arg)
{
    OS_ERR err;
    CPU_SR_ALLOC();
    p_arg = p_arg;

    CPU_Init();
    #if OS_CFG_STAT_TASK_EN > 0u
        OSStatTaskCPUUsageInit(&err); //统计任务
    #endif

    #ifndef CPU_CFG_INT_DIS_MEAS_EN //如果使能了测量中断关闭时间

```

```

    CPU_IntDisMeasMaxCurReset();
#endif

#if OS_CFG_SCHED_ROUND_ROBIN_EN //当使用时间片轮转的时候
    //使能时间片轮转调度功能,设置默认的时间片长度 s
    OSSchedRoundRobinCfg(DEF_ENABLED, 10, &err);
#endif

    OS_CRITICAL_ENTER(); //进入临界区
    //创建 LED1 任务
    OSTaskCreate((OS_TCB *) &Led1TaskTCB,
                (CPU_CHAR *) "led1 task",
                (OS_TASK_PTR) led1_task,
                (void *) 0,
                (OS_PRIO) LED1_TASK_PRIO,
                (CPU_STK *) &LED1_TASK_STK[0],
                (CPU_STK_SIZE) LED1_STK_SIZE/10,
                (CPU_STK_SIZE) LED1_STK_SIZE,
                (OS_MSG_QTY) 0,
                (OS_TICK) 0,
                (void *) 0,

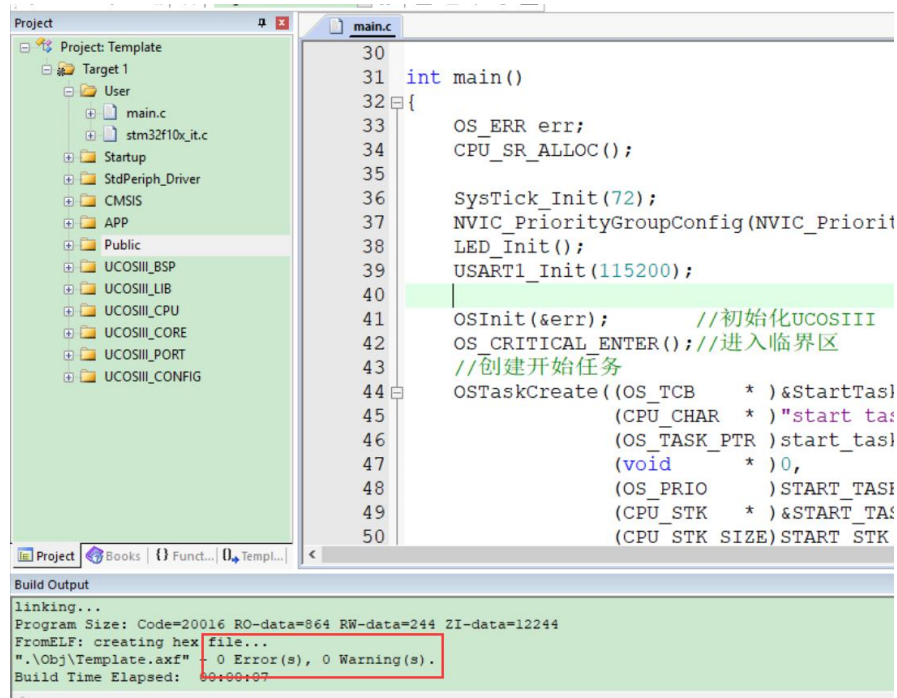
                (OS_OPT) OS_OPT_TASK_STK_CHK|OS_OPT_TASK_STK_CLR,
                (OS_ERR *) &err);

    OS_CRITICAL_EXIT(); //进入临界区
    OS_TaskSuspend((OS_TCB*) &StartTaskTCB, &err); //挂起开始任
务
}

//led1 任务函数
void led1_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    while(1)
    {
        LED1=!LED1;
        OSTimeDlyHMSM(0, 0, 0, 200, OS_OPT_TIME_HMSM_STRICT, &err); //延
时 200ms
    }
}

```

编写完成后，编译下工程，没有错误和警告，如图所示：



UCOSIII 中使用的 API 函数大家可以参考光盘“\8—STM32 相关资料\STM32 深入学习资料\UCOS 学习资料\UCOSIII 资料”《uCOS-III 中文翻译》文档，里面面对 UCOSIII 有着非常详细的介绍，包括 UCOSIII 使用的 API 函数。

1.3 实验现象

将编译好的程序下载到开发板内可以看到，D1 指示灯间隔 200ms 闪烁，证明 UCOSIII 移植到 STM32F1 上成功。

课后作业

(1) 按照移植教程，自己动手移植一次，如果在移植过程中遇到问题，请查看本教程或移植模板。